Deep Learning Aided Sensor Fusion for Drift Reduced IMU Orientation Estimation

Rahul Verma
201629064

Supervisor: Professor. Zhiqiang Zhang
Assessor: Dr. Chris Wood

A project presented for the degree of
Masters of Engineering in Electronic Engineering

School of Electrical and Electronic Engineering
University of Leeds

School of Electronic and Electrical Engineering
FACULTY OF ENGINEERING

UNIVERSITY OF LEEDS

ELEC5870M MEng Individual Project

Declaration of Academic Integrity

Plagiarism in University Assessments and the Presentation of Fraudulent or Fabricated Coursework

Plagiarism is defined as presenting someone else's work as your own. Work means any intellectual output, and typically includes text, data, images, sound or performance.

Fraudulent or fabricated coursework is defined as work, particularly reports of laboratory or practical work that is untrue and/or made up, submitted to satisfy the requirements of a University assessment, in whole or in part.

Declaration:

- I have read the University Regulations on Plagiarism[1] and state that the work covered by this declaration is my own and does not contain any unacknowledged work from other sources.
- I confirm my consent to the University copying and distributing any part or all of my work in any form and using third parties (who may be based outside the EU/EAA) to monitor breaches of regulations, to verify whether my work contains plagiarised material, and for quality assurance purposes.
- I confirm that details of any mitigating circumstances or other matters which might have affected my performance and which I wish to bring to the attention of the examiners, have been submitted to the Student Support Office.

[1] Available on the School Student Intranet

Student Name: Rahul Verma Date: 18/05/2026

Signed: Rahul Verma

Abstract

Inertial Measurement Units (IMUs) are widely used in a variety of applications, such as Body Sensor Networks (BSNs) for orientation estimation. However the gyroscope suffers from drift due to sensor bias and noise that when integrated accumulates over time. This project investigates a deep learning-based, approach which aims to mitigate gyroscopic errors which can be integrated with sensor fusion techniques, to achieve more accurate orientation estimates. The proposed deep-learning architecture leverages both neural networks and a temporal history to learn complex and non-linear error patterns in IMU data, exploring if it outperforms a Manifold Unscented Kalman Filter (MUKF) without learned corrections. The network outputs a correction for the incoming gyroscope sample, and the data used in training, testing and validating the model comes from public datasets, such as the Berlin Robust Orientation Estimation Assessment Dataset (BROAD). This project summarises that a deep-learning model can be used to provide meaningful corrections to gyroscope measurements and is effective in achieving more accurate orientation estimates.

Acknowledgements

I would like to express my sincere gratitude to Professor. Zhiqiang Zhang for his continuous support and guidance throughout this project. Professor. Zhang's insights and advice during this project, in particular the explanation of quaternion algebra and in-depth explanation of filters, were invaluable in shaping the direction of my work. In addition, offering his personal copy of Body Sensor Networks to aid in my understanding was highly appreciated.

I would also like to acknowledge the authors and maintainers of the publicly available datasets of BROAD. Their efforts in collecting and labelling high quality IMU data, and optical motion capture ground truth enabled the training, testing, and validation of the neural network. I am grateful for their contributions and cite them accordingly in this report.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Inertial Measurement Units and their applications

IMUs are composed of multiple sensors which include a gyroscope, accelerometer, and occasionally a magnetometer. These three sensors measure the angular rate, specific force, and the local magnetic field vector respectively. These measurements can be used to estimate the orientation of an object through integration of the angular rate. This data acquisition is essential for several applications such as BSNs[1], robotics, and autonomous vehicles where these systems rely on high-rate orientation updates.

IMUs have emerged as a key technology due to their ability to work in a self-contained environment. In environments where external references of orientation are unavailable or unreliable, technologies such as IMUs are attractive. Filters, such as the KF, allow the use of accelerometers and magnetometers to guide and correct state estimation, but these also suffer from failures like magnetic disturbances, and high linear accelerations. Due to these limitations, there is significant motivation to explore deep-learning methods that compensate for these conditions and errors.

1.2 Problem Statement: IMU Drift and Its Impact

IMUs have shown promise in determining the orientation of an object in motion. However, IMUs suffer from a limitation called drift. IMU drift is characterised by the accumulation of errors through the integration of the angular rate. The sources of these errors include constant bias, scale factor errors, and others expanded in section 3.1.2. Errors are not exclusive to the gyroscope but also affect the accelerometer and magnetometer. Drift is also dependent on the type of IMU that is used. Lower cost/grade IMUs suffer from drift at a higher magnitude, which results in orientation inaccuracies much quicker compared to higher cost/grade IMUs. Errors that lead to inaccuracies in the orientation estimation of an object determined by the IMU.

Kianifar et al. explored using IMUs for orientation estimation in a clinical setting. They found that for rotation angles parallel to gravity, drift due to gyroscope bias cannot be compensated by the accelerometer[2]. Even with multiple sensors, it is still challenging to find an accurate orientation estimation. Thus it is important to try and address the orientation problem by addressing gyroscopic drift.

Consequently, this project aims to address gyroscopic drift by using deep-learning methods to learn the complex and non-linear nature of biases and errors.

1.3 Research Question and Hypotheses

The question this project aims to answer is: **"Can Deep Learning be used to learn the drift pattern to get drift reduced orientation estimation?"** While this is the overarching question, it can be broken down to the following:

- How effectively can deep learning learn the drift experienced?
- Can this model generalise to unseen conditions?
- What are the advantages or disadvantages of using Deep Learning compared to traditional approaches?

The hypothesis that I state of this project is that by incorporating a deep-learning model, we will be able to see a percentage decrease in the orientation error over a period of time.

2 Societal Factors

2.1 Healthcare & Rehabilitation

In healthcare and rehabilitation, there is a wide need for orientation estimation from limb kinematics to assessment protocols for Axial Spondyloarthritis [3]. However, these studies also highlight orientation drift due to IMUs as a limitation to their uses.

Franco et al, highlights that IMUs have not widely been used for orientation estimation in a clincal setting due to gyroscopic drift [3]. Specifically, the study conducted range of motion tests that uses an IMU and Kalman Filter under constrained movements. The BASMI movements performed and recorded all spanned for approximately 30s and achieved and MAE around 4.0 degrees in their tests [3]. However, all of these trials were performed in a controlled laborartory setting with constrained movements to limit the accumulation of drift. Further, this sensor alignment assumes that the sensors reflect the actual tilt of the spine in the sagittal and frontal planes, and that the spine axial rotation is zero at t0 [3]. Under these constraints and assumptions, trying to extend this protocol outside of the laborartory and into clinical settings which have a lot of magnetic disturbances or under continuous monitoring of uncontrolled motions, their approach breaks down and drift accumulates. The proposed solution in section 6.6 serves as a way of extending and enabling further investigation of range of motion under non-constrained and continuous monitoring. However, it is important to acknowledge that this proposed solution is based on data found in section 4 compared to spinal kinematics data. However, the dataset does include slow small rotations which is representative of the types of motions that Franco et al performed.

2.2 Agriculture

3 IMU Basics and Operational Principles

3.1 Gyroscope: Operations and Errors

The gyroscope is the main component that is used to determine the orientation of the object. It measures the angular rate in its orthogonal axes $\omega^x, \omega^y, \omega^z$ which is used to determine the object's orientation at a discrete time. The orientation is determined through the integration of angular rate. In this chapter, the quaternion representation is selected due to being more computationally efficient than other methods [4] and does not suffer from gimbal lock [5]. Gimbal lock is described as a phenomenon where two of the three rotational axes of an object align causing a loss of one degree of freedom (DOF), since the Euler angle representation becomes singular.

3.1.1 Angular Rate Integration

We can define a quaternion that is represented by the angular rate, where ω is the real angular rate, and $\omega_q(t)$ is the pure quaternion. Note: Both the attitude and updated quaternions should be normalised to ensure quaternions represent rotations. $q_k \leftarrow \frac{q_k}{\|q_k\|}$.

$$\omega_q(t) = [0,\ \omega_x(t),\ \omega_y(t),\ \omega_z(t)] \tag{1}$$

The orientation evolves according to [5]

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_k \tag{2}$$

where Δq_k refers to the Special Orthogonal group 3 (SO(3)) exponential map, which maps rotational vectors to incremental rotation quaternions [5].

$$\Delta q_k = \begin{bmatrix} \frac{\omega}{|\omega|} \sin \frac{|\omega|\Delta t}{2} \\ \cos \frac{|\omega|\Delta t}{2} \end{bmatrix} \tag{3}$$

This was all done by using an ideal ω_k, if we were to model the angular rate from a gyroscope as [6]

$$\omega_{m,k} = \omega_k + b_k + n_k \tag{4}$$

where $\omega_{m,k}$ is the measured angular rate, ω_k is the true angular rate, b_k is the bias term, and n_k is the noise term. The real quaternion orientation update is as follows where Δq_k becomes $\Delta q_{m,k}$.

$$\Delta q_{m,k} = \begin{bmatrix} \frac{\omega_{m,k}}{|\omega_{m,k}|} \sin \frac{|\omega_{m,k}|\Delta t}{2} \\ \cos \frac{|\omega_{m,k}|\Delta t}{2} \end{bmatrix} \tag{5}$$

and the update is now

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_{m,k} \tag{6}$$

This shows that the inclusion of the bias and noise accumulates over samples as we move forward to the next k, q_{k-1} will incorporate the previous error terms. This results in an accumulated orientation error.

For more information about this derivation see Chapter 12[5].

3.1.2 Gyroscope Error Sources

Constant Bias

The bias of a gyroscope is the average output from the gyroscope when it is not undergoing any rotation [7]. This is measured in $^\circ/h$ and can be estimated by taking an average of the output. As this bias is constant, the drift it causes grows linearly with time. However, as discussed further in this section other error sources can make this difficult to determine. Figure 1 shows how constant bias changes the output.

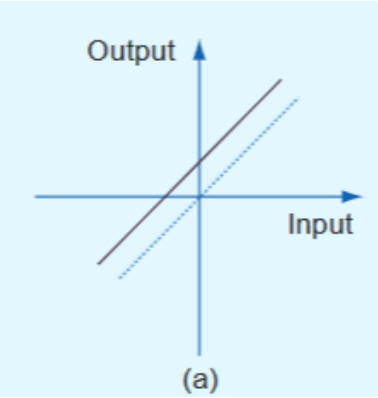


Figure 1: Bias Effect on Gyro [8]

White Noise / Angle Random Walk (ARW)

The gyroscope is also affected by some white noise [7]. This white noise sequence is zero-mean uncorrelated random variables between samples and across axes. When the gyroscope signal is integrated to obtain an angle, this white noise produces an ARW. The units of ARW are denoted by $^\circ/\sqrt{h}$, this shows that the deviation of angle error grows proportionally to $\sqrt{t}$. As ARW is due to random variables, it is classified as the 1σ of the orientation error. Analog Devices shows this in figure 2, where the ARW is $0.17^\circ/\sqrt{h}$.

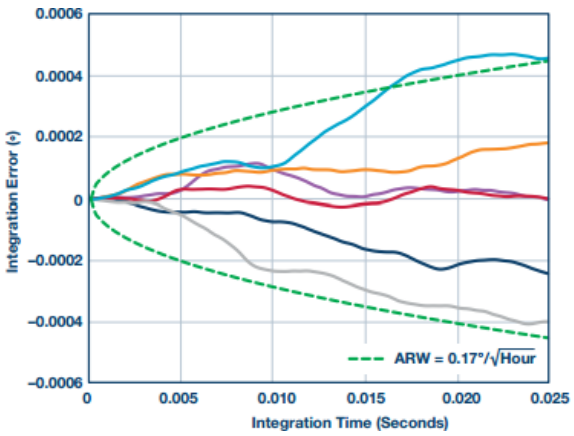


Figure 2: Effect of ARW on Gyro Output [9]

Temperature

Temperature fluctuations due to changes in the environment and sensor heating induce a movement in the bias, this relationship is also highly non-linear [7]. Therefore, it can be very difficult to model and subsequently subtract from the gyroscope measurements compared to a constant bias where temperature is constant.

For more gyroscope error sources and effects, see [7].

3.2 Accelerometer: Operations and Errors

The accelerometer is another component used in determining an orientation estimate of an object. It measures the specific force along its orthogonal axes a^x, a^y, a^z. The main purpose of the accelerometer is to offer a gravity reference to the system such that an accelerometer can provide drift-free inclination estimates [10]. However, the accelerometer, as stated above, does not only measure a gravity reference but also any linear accelerations that the object also experiences. Under conditions of high accelerations, the reference is unreliable and is only reliable for static or slowly moving objects [11]. Additionally, the accelerometer cannot detect rotations about the gravity vector, so it cannot provide a yaw/heading of

the object. However, like the gyroscope, it also suffers from a variety of errors which causes instability in the output.

3.2.1 Accelerometer Error Effects

Accelerometers suffer from very similar sources of errors to the gyroscope (section 3.1.2). Therefore, in this section we will discuss the implications of these errors for our orientation estimates.

Constant Bias

The effect of constant bias is that it causes an offset in the roll/pitch of the system. Unlike constant bias in the gyroscope, it does not grow linearly with time but is a constant offset irrespective of time elapsed.

High Linear Acceleration

As discussed above, the largest problem is high linear accelerations which corrupts the gravity reference. This is a dominant issue in high dynamic motions as the accelerometer will not be able to correct the orientation estimation through fusion, especially if gyroscopic drift is not addressed.

3.3 Magnetometer: Operations and Errors

Magnetometers measure the local magnetic field along its orthogonal axes m^x, m^y, m^z. The magnetometer can be used to determine the heading (yaw) of an object. Sensor fusion/filtering algorithms are dependent on sensing the local magnetic field to eliminate drift in the azimuth (yaw) portion of orientation estimates [12]. It does this by assuming that the measured local field vector provides a reference relative to Earth's field. However, changes in the local field due to other field sources, can distort this measurement which gives rise to erroneous heading corrections.

3.3.1 Magnetometer Error Sources and Effects

Hard-Iron / Soft-Iron

Hard-iron is a constant additive magnetic field applied to the sensor due to the interaction of magnetic fields [13]. This interaction between these sources leads to a constant vector being applied to the measurement.

Soft-iron effects are generated by the interaction of an external magnetic field on any ferromagnetic materials near the sensor [13]. The resulting magnetic field depends on the magnetic field vector with respect to the orientation of the soft iron material [14].

The equations below show how hard-iron and soft-iron effect the magnetic field [13].

$$h_{SI} = C_{SI} R_E^B h^E \tag{7}$$

$$h_{SI+HI} = h_{SI} + b_{HI} \tag{8}$$

where h_{SI} is the soft-iron magnetic field, $C_{SI} \in R^{3\times3}$ the soft-iron matrix, $R_E^B \in SO(3)$ the Earth to body rotation matrix, h^E the Earth magnetic field, and b_{HI} the hard-iron bias.

The above discussion on hard-iron/soft-iron effects assumes these disturbances are static relative to the sensor. However, in many real operating conditions, these magnetic fields vary which corrupts the heading reference to a greater degree.

4 Datasets

4.1 Dataset Selection

4.1.1 BROAD

Feature	Description
Sampling Rate	286 Hz
Ground Truth	Optical Motion Capture (OMC) quaternion $[w, x, y, z]$ in ENU reference frame
Number of Trials	39
Number of Samples	1508364 samples
Diverse Trial Set	slow/fast rotations, translations, magnetic disturbances

Table 1: Key characteristics of the BROAD dataset [15]

BROAD [15] is the dataset used in the project. It has several characteristics that make it suitable for selection. First of these is the inclusion of a magnetometer, this project specifically focuses on 9 DOF IMUs, which causes many other datasets to be excluded based on this alone. The OMC ground truth quaternions provide a high accuracy reference which make it a suitable training target for deep learning. The ground truth has also been synchronised with the IMU readings, allowing for supervised learning at every timestep without discontinuities or needing to interpolate.

Additionally, BROAD trials include slow and fast rotations, isolated accelerometer and magnetometer disturbances, and combined scenarios. This means that a variety of trial types across the set can be split such that train, validation, and test sets are equally represented. Hence, the evaluation of the deep learning method reflects generalisation across disturbance conditions. Furthermore, BROAD includes rest phases for each trial, which allows for sensor noise characteristics to be determined empirically. This is expanded upon in table 3.

4.1.2 Simulated Data

Simulators (MATLAB's Navigation Toolbox [16]), can be used to generate a diverse IMU dataset with the advantage of near perfect ground truth quaternions. This is due to the ability of creating trials that focus on different scenarios and define the motion profile the IMU undergoes. However, they suffer from major restrictions that do not make them suitable.

The fundamental problem is that the real sensor characteristics are hard to model as these simulations mainly use simple Gaussian processes. Real noise is more complex with bias being temperature dependent, noise is correlated across axes, and magnetic disturbances vary both spatially and temporally. This makes deep learning models trained on this data harder to generalise to real scenarios (sim to real gap) as the data does not capture the same level of complexity in error sources described in section 2.1.2. This domain shift causes a model's evaluation on real IMUs to underperform.

Moreover, simulation causes leakage into deep learning design decisions as simulated sensor noise profiles are known beforehand. This leads to any decisions made to be circular in nature.

Due to the main restriction of the sim to real gap, the choice was made to use the BROAD dataset.

4.1.3 Train, Validation, and Test Dataset Split

Table 2 shows the train, validation, and test split of the BROAD dataset. The main motivation for the test split is that it spans four distinct trial types being that both robustness under very disturbed scenarios and accuracy under undisturbed but high dynamics are tested. This is due to the fact that the MUKF alone would struggle to correct under these conditions, giving a reasoning to why the proposed solution is needed. Moreover, the entirety of each trial only resides in their individual split, this is because splitting trials mid-sequence would risk data leakage across the splits. Likewise, the validation set also includes similar trial scenarios as the test set so that it can influence design decisions made in section 6.4.

Split	Trial	Motion Type
Test (5)	9	Undisturbed, fast rotation
	25	Disturbed, tapping (accelerometer disturbance)
	31	Disturbed, stationary magnet
	34	Disturbed, attached magnet (1-5 cm)
	39	Mixed (disturbed and undisturbed)
Validation (6)	7	Undisturbed, fast rotation
	16	Undisturbed, fast translation
	20	Undisturbed, slow combined
	27	Disturbed, vibrating smartphone
	29	Disturbed, stationary magnet
	35	Disturbed, attached magnet (1-5 cm)
Train (28)	1-6, 8, 10-15, 17-19, 21-24, 26, 28, 30, 32, 33, 36-38	Mix of undisturbed (slow/fast rotations, translations, combined) and disturbed conditions

Table 2: Train, validation, and test split of the 39 BROAD trials [15]. Test trials were selected to represent worst-case disturbance conditions across accelerometer and magnetometer disturbances, as these are the conditions most challenging for sensor fusion alone.

The justification of the three splits is that the model only ever sees the train split, it is validated on the validation set, and the best model is only ever evaluated once all training and hyperparameter decisions have been finalised (test set). This is so any design decisions and changes only ever depend on the validation set as to prove generalisation the test should never be able to influence model design decisions and model performance is an unbiased estimate of generalisation.

4.2 Scope and Limitations of BROAD

The BROAD dataset is not without its limitations, the main limitation is that it contains a small number of distinct trials, which limits the diversity of conditions the model can be exposed to. In comparison, most conventional deep learning tasks such as natural language processing and image processing tend to have more diverse pool in its training set which allows for their solutions to generalise and converge easier. This limitation can lead to poorer generalisation to our test set and thus may reduce performance on unseen conditions. Hestness et al empirically demonstrates this through a power-law which states that model performance increases as the training set increases [17]. However, adding noise augmentation (section 4.5) can artificially diversify the training set and is able to alleviate this limitation but not fully solve it.

Further, the data collected was in a controlled laboratory setting, where clean and predictable conditions are present. In a real deployment, temperature, and scale factor errors may be more present compared to the dataset. This can hurt generalisation even if the same hardware is being used. However, to try and alleviate this, the model was tested on the more disturbed conditions (see section 4.1.3) to try and test the model at these extreme conditions.

4.3 Data Normalisation

Data normalisation is the process of converting a variable's actual range to a standard range of values that is seen at a neural network's (NN) input. LeCun et al highlights normalisation by stating that convergence is usually faster if the average of the input variable over training is close to zero [18]. This is due to any shift in the average input from zero will bias NN weight updates in a certain direction be it positive or negative. Meaning, that if the network wanted to shift weights in the opposite direction it must 'zigzag' which is inefficient and slow [18]. Sola et al expands on normalisation stating we can expect better predictions from our NN by utilising normalisation. It was concluded that with adequate normalisation of input data, estimation errors can be reduced by a factor of by a factor between 5-10 [19].

Thus, data normalisation was considered to be a very important addition to the pre-processing pipeline.

4.3.1 Normalisation Implementation

The proposed solution uses Z-score normalisation for its method [20]. This is because it satisfies the requirements laid out by LeCun et al [18] for zero mean inputs preventing weight updates being biased in a certain direction and reducing estimation error [19].

Given a raw input signal x, the Z-score normalised signal $\tilde{x}$ is computed as:

$$\tilde{x} = \frac{x - \mu}{\sigma} \tag{9}$$

where μ is the per-axis mean and σ is the per-axis standard deviation, both computed from the training set only:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i \tag{10}$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2} \tag{11}$$

where N is the total number of samples in the training set. This is applied independently to each of the nine input channels, as each sensor axis operates in a different physical unit and dynamic range.

These normalisation factors are only computed on the training set and applied to all inputs of the training, validation, and test sets. This is because whilst training the model should not be able to infer any information from the test and validation sets. This would consitute as data leakage, as test and validation statistics would influence the normalisation applied at training.

Other methods like min/max scaling was also considered, this method scales all data between a range like $[-1, +1]$. However, this method is not suitable in this application, due to trials like disturbed tapping, where if only a small section of the trial experiences high transient motions, these samples would be the min or max and the rest of the samples are compressed around the mid point (e.g. 0).

4.4 Window Sampling

A window sampling strategy was implemented in this project to address a limitation of the BROAD dataset, this being its limited trials. The strategy employed was random window sampling on the train set, this selects a random window of samples within a trial to be the input to the model. This allows the model to see not only different trials randomly with PyTorch using "shuffle=True" [21], but different windows within the file. This results in a trial of length N having $N - T$ possible windows across training where $T = windowlength$. This improves generalisation and therefore model performance as the model sees a wide variety of windows within each trial, in tandem with section 4.5, this should be able to alleviate the main limitation of the dataset.

The validation and test sets however, do not employ random window sampling but deterministic and non-overlapping windows. This is because it means consistent evaluation across each epoch. If it were randomised then each epoch would return different validation losses meaning that they are not comparable to another epoch. Furthermore, this ensures full coverage across the validation and test sets as each sample is evaluated exactly once, this is so the validation loss is not biased across the trials.

4.5 Noise Augmentation

Noise augmentation was added to the dataset to address one of the main limitations of the BROAD dataset, being that it contains a small number of trials which limits the number of samples that the model can be trained on. Mentioned in section 4.2, limited samples can hurt generalisation and model performance. Thus it was considered to be pertinent that the dataset is extended. Per axis noise standard deviations is provided in the BROAD dataset [15] which was measured through rest phases. Noise is per axis and not isotropic due to individual axes having different noise characteristics (see table 3).

Sensor	σ_x	σ_y	σ_z
Gyroscope	$0.10\,^\circ/\mathrm{s}$	$0.09\,^\circ/\mathrm{s}$	$0.12\,^\circ/\mathrm{s}$
Accelerometer	$0.044\,\mathrm{m/s^2}$	$0.050\,\mathrm{m/s^2}$	$0.074\,\mathrm{m/s^2}$
Magnetometer	$0.71\,\mu\mathrm{T}$	$0.70\,\mu\mathrm{T}$	$0.68\,\mu\mathrm{T}$

Table 3: Per-axis noise standard deviations derived from a 49-minute stationary recording [15].

Noise augmentation is only added to the training set of data, as this is the set that needs to be expanded. If it were added to the validation, or test set then evaluation would be inconsistent and not reproducible.

However, due to section 4.3, the noise standard deviations must also be normalised. This is because after normalisation the inputs to the NN are no longer represent physical signals. This means that to correctly add noise, the standard deviations also must be normalised.

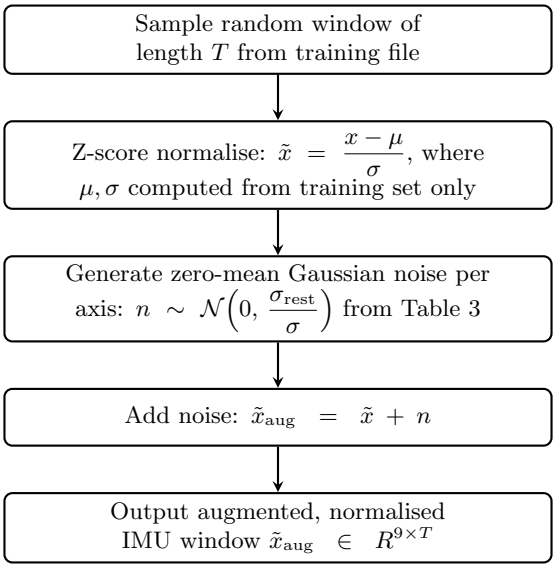


Figure 3: Noise augmentation pipeline applied to each training sample. The noise standard deviations are derived from the rest-phase measurements (Table 3) and divided by the signal standard deviation σ to account for Z-score normalisation. Augmentation is applied only during training.

5 Manifold Unscented Kalman Filter

5.1 Kalman Filter Overview

The Kalman filter (KF) is an optimal recursive estimator used to estimate a state under noisy measurements and uncertain dynamics. It works under an assumption that the relationship between our state and measurements can be modelled as linear and Gaussian [22].

Term	Description
State Vector ($\mathbf{x}$)	The quantity the filter is trying to estimate.
State Estimate Covariance ($\mathbf{P}$)	The uncertainty the filter has in its state estimation.
Process Noise Covariance ($\mathbf{Q}$)	The uncertainty the filter has in its process model, i.e. how much the filter trusts the dynamics used to propagate the state.
Measurement Noise Covariance ($\mathbf{R}$)	The uncertainty the filter has in its measurement values, used to correct the state.
Kalman Gain ($\mathbf{K}$)	Balances the weight given to the process model prediction versus the incoming measurement.

Table 4: Kalman Filter Terms and Definitions

The filter implements a two step process of prediction and update. The predicition propagates our state estimation and the update corrects the state estimation using one or multiple measurements. The covariance P is updated by the K after every new sensor value to reflect the new information the filter has gained about the system.

However, the KF breaks down for orientation estimation due to some key factors. First, quaternion kinematics are non-linear (see equation 3) and the assumption that the relationship is linear is violated. Second, as the relationship is non-linear, this breaks the Gaussian noise process model as propagation through a non-linear function produces a non-Gaussian output. Third, as quaternions exist in SO(3) and not Euclidean space, adding a Gaussian term to the state can push it off the manifold [23].

5.2 Unscented Kalman Filter

The Unscented Kalman Filter (UKF) is an extension of the KF that allows for non-linear relationships to be more accurately modelled by the filter. Traditionally, to apply a KF to a non-linear relationship an extended KF (EKF) is used which linearises all non-linear models through the use of Jacobian matrices that are evaluated at the current state estimate [24]. However, better performance tends to be achieved by a UKF for orientation estimation as a UKF gives more accurate covariance propagation at larger rotations. This is due to UKFs being able to capture second order non-linear effects through the use of sigma points [24] [25].

Sigma points are drawn from a specific, deterministic algorithm that when a non-linear function is applied to these points; a set of $2n+1$ transformed points are left. These sigma points are then used to reconstruct the mean by taking a weighted average of these points and the weighted outer product for the covariance.

However, with the non-linear propagation addressed, computing a weighted mean of quaternions directly is geometrically invalid. This is due to quaternions exisiting on the SO(3) manifold, as the element-wise average of quaternions does not lie on the unit sphere and therefore does not represent a valid rotation.

For an in-depth derivation regarding the UKF see [24].

5.3 The Manifold Problem

The core issue of the UKF is that it assumes that the x lies within Euclidean space, however quaternions lie on the SO(3) manifold. So computing the mean and adding the measurement noise to the quaternion state is geometrically invalid, thus the state that is estimated is also invalid.

The solution to this problem is to perform any arithmetic computations that the UKF requires within the local tangent space (so(3)). By taking the current state error estimate and mapping it to and from Euclidean vector space, the predict and update steps are performed validly and the geometric

representation of the state is conserved. Brossard et al and Crassidis et al explore and show this within their UKF-Manifold and Unscented Quaternion Estimator (USQUE) implementations [26] [27].

The equations below show how to map a quaternion on the SO(3) manifold to local tangent space so(3) [28].

$$\theta = 2 \arccos\left(|q_w|\right) \tag{12}$$

$$\operatorname{sgn}(q_w) = \begin{cases} +1 & \text{if } q_w \geq 0 \\ -1 & \text{if } q_w < 0 \end{cases} \tag{13}$$

$$\log(\mathbf{q}) = \begin{cases} 2 \cdot \operatorname{sgn}(q_w) \cdot \mathbf{q}_v & \text{if } \theta \leq \epsilon \\ \dfrac{\theta}{\sin(\theta/2)} \cdot \operatorname{sgn}(q_w) \cdot \mathbf{q}_v & \text{otherwise} \end{cases} \tag{14}$$

Thus the implementation used in this project is based on UKF-M as it uses the Fréchet mean, which is uses both equations 3 and 14. This avoids the need for additional Rodrigues parameter representation [26] [27].

5.4 Filter Design

Table 3 below shows the configuration used for the MUKF.

Parameter	Value	Description
State	$\mathbf{q} \in SO(3)$	Unit quaternion $[w, x, y, z]^\top$
Error state	$\delta\boldsymbol{\phi} \in R^3$	Rotation vector in tangent space $so(3)$
Sigma points	$2n + 1 = 7$	$n = 3$, spread parameter $\kappa = 1$
Process noise $\mathbf{Q}$	$\operatorname{diag}(\boldsymbol{\sigma}_\omega^2) \cdot \Delta t$	$\boldsymbol{\sigma}_\omega$ from BROAD rest-phase measurements
Accel. noise $\mathbf{R}_{\text{acc}}$	$\operatorname{diag}([0.044,\ 0.050,\ 0.074]^2)$ m^2/s^4	From BROAD rest-phase measurements
Mag. noise $\mathbf{R}_{\text{mag}}$	$\operatorname{diag}([0.71,\ 0.70,\ 0.68]^2)$ μT^2	From BROAD rest-phase measurements
Accel. gate	2.5% of $\|\mathbf{g}\|$	Rejects updates under dynamic acceleration
Mag. gate	30% of $\|\mathbf{m}_{\text{ref}}\|$	Rejects updates under magnetic disturbance
Sampling rate	286 Hz	Matches BROAD dataset

Table 5: MUKF configuration

5.4.1 State Representation

The state represents the quaternion in SO(3) and the error state is the uncertainty the MUKF has in its orientation estimate. This is a significant deviation compared to the interim implementation. This is because the deep-learning model is used to correct the gyroscope signal, meaning that if the filter also tried to estimate this error in the gyroscope signal there would be a conflict between the filter and model. The 3D error state lives in local tangent space (so(3)), this is so the predict and update steps that need to happen in Euclidean space can be performed correctly and then mapped to SO(3) using equation 3.

5.4.2 Process Model

First, sigma points are generated from the current state estimate. For our case $n = 3$ for a 3D error state, giving 7 sigma points. Then each sigma point is propagated through our process model, which is the equation 3. This is the predict step of the filter and gives us the next predicted state of the filter (quaternion estimate). After propagating the sigma points, the predicted mean is recovered via the Fréchet mean [26], and the predicted covariance is reconstructed from the weighted outer product of the tangent space errors (3D errors).

5.4.3 Measurement Models

The measurement models are the accelerometer and magnetometer that are used to update and correct the state estimate from the prediction. The accelerometer update compares the measured specific force against the gravity vector (ENU frame) rotated into body frame. As mentioned in section 3.2, this corrects the tilt (roll/pitch) of the state estimate. The magnetometer is compared against the horizontal magnetic reference which is collected over the first 50 samples which is in the rest phase [15]. This is so the filter has a clean magnetic reference before any disturbances occur. Again as mentioned in section 3.3 this corrects the azimuth/yaw of the current filter estimate.

6 Deep Learning Architecture

6.1 Aims of the Deep Learning Model

The motivation for learning $\Delta\boldsymbol{\omega}_t = [\Delta\omega_x, \Delta\omega_y, \Delta\omega_z]^\top$ comes from the fact that while sensor fusion techniques of downweighting or skipping of the accelerometer and magnetometer when they are unreliable, drift cannot be mitigated due to the accumulation of gyroscopic bias and noise [29][30]. Moreover, learning a correction to be applied to the gyroscope signal is an easier task than producing a correct measurement. This is because instead of the model needing to predict the entire signal, including its dynamics of motion, the model just applies a correction. This explaination is further explored in section 6.3.4.

6.2 Model Selection

6.2.1 Recurrent Neural Networks (RNNs)

The first type of model that was considered is a RNN. RNNs stem from an idea of recurrence, that is that the output depends on some input at time t and an internal state of the model that holds information about previous outputs of the model. The general idea can be described by the equation

$$\hat{y} = f_{model}(x_t, h_{t-1}) \tag{15}$$

where $\hat{y}$ is the output, f_{model}, is a function that describes the model, x_t is the input at time t, and h_{t-1} is a vector state that describes the internal state of the model at the previous timestep. Expanding on this, the state of the model is updated as follows

$$h_t = f_w(x_t, h_{t-1}) \tag{16}$$

where f_w is a function with weights w. This expands to the equations

$$\hat{y} = W_{hy}^T h_t \tag{17}$$

$$h_t = \phi(W_{hh}^T h_{t-1} + W_{xh}^T x_t) \tag{18}$$

where $W_{hh}^T, W_{hy}^T, W_{xh}^T$ are a set of weight matrices that are reused for every timestep, and ϕ represents an arbitrary activation function.

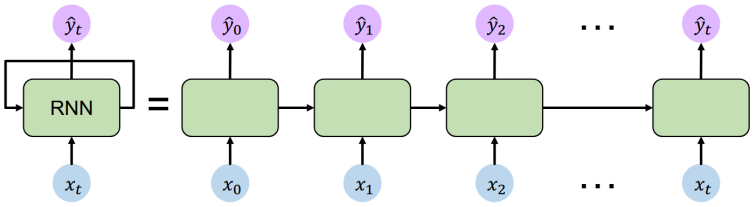


Figure 4: RNN structure [31]

Figure 4 shows the unrolled flow of the RNN, where the arrows between each block represents the passing of h_t, from one input to the next. The point of the recurrence state h_t, is to show how the output of the model depends on context from previous inputs plus h_t. This is especially relevant in our case, as described in section 3.1.1, the accumulation of gyroscopic errors comes from the integration of the angular rate. As these errors accumulate, the current error depends on the history of the gyroscope measurements.

There are many problems with RNNs however, the biggest being the problem of exploding/vanishing gradients. RNNs are trained via backpropagation through time, which propagates gradients across timesteps. For long sequences, computing the gradient with respect to h_0 involves many factors of repeated multiplication of weight matrices across time. This can lead to many values of the computation being greater than 1, which explodes gradients leading to unstable training. In the case, where many values are less than 1,

this causes vanishing gradients where learning long-range dependencies becomes difficult. Additionally, as the hidden state h_t is propagated sequentially through time, when performing backpropagation, training cannot be parallelised over timesteps. This results in high training times compared to convolutional models.

Updated versions of the RNN have been proposed to solve the problem of vanishing and exploding gradients. These being Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs). These methods exploit the use of a cell state that does not need to perform repeated multiplication of weight matrices, however, they are still subject to long training times and the gradient problem is not fully solved but mitigated [32][33].

6.2.2 Convolutional Neural Networks (CNNs)

The next type of model considered was the CNN, even though typically it is used in computer vision tasks; it has seen wide use in research relating to drift reduction [34][35]. This model is based on the idea of convolving input data with a learnable filter to extract local features in the input data. The filter can be thought of like a window that slides over the data, and each filter has a defined set of weights in which it multiplies with input data. This convolution operation can be described by the equation below:

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t + k - \lfloor K/2 \rfloor] \tag{19}$$

$$y_f[t] = \phi(z_f[t]) \tag{20}$$

where $y_f[t]$, is the output of filter f, b_f is a bias, C_{in} is the input channels (for our case C_{in} would be 3 for $\omega_x, \omega_y, \omega_z$), K is the kernel/filter width, $w_{f,c}$ is the weights of the filter for a specific channel, x_c is the input data of that channel, and ϕ is an arbitrary non-linear activation function (typically ReLU).

For example if $K = 3$, the convolution will occur over the timesteps $x[t-1], x[t], x[t+1]$, this is how the convolution extracts local features in the input data.

Typically, these CNNs are made up of layers, where in each layer the number of filters grows e.g. $L1 = 16, L2 = 32, L3 = 64$ etc. These additional filters in later layers allow the model to learn more complex feature sets based on the previous layers' filter. Zeiler et al. [36] shows this in their figure 2 where their convolutional network is used for image classification.

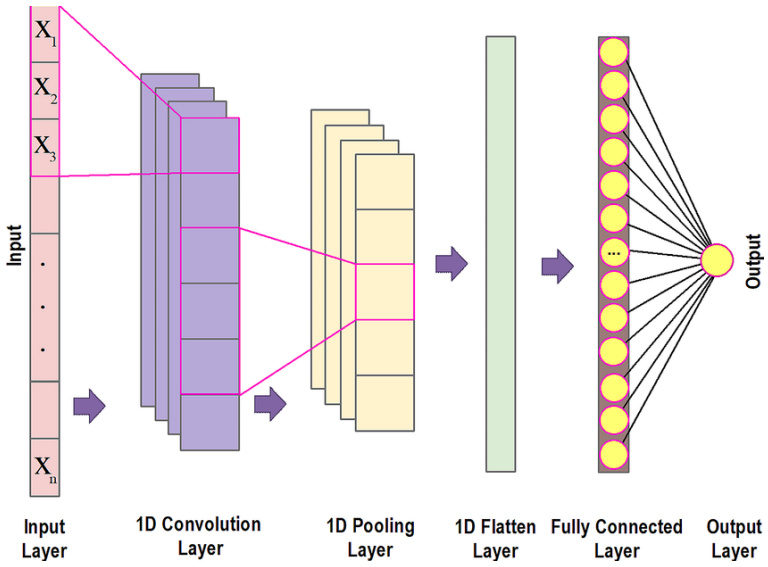


Figure 5: 1D CNN structure [37]

CNNs are more attractive than RNNs due to its ability to be efficiently trained by using parallelisation [38]. This leads to the ability to test multiple different iterations of a model where parameters and loss functions may be changed to explore what effects they may have. CNNs also do not suffer from vanishing/exploding gradients as much and are seen to be more stable during training due to no backpropagation through time.

However, unlike RNNs, baseline CNNs cannot produce outputs due to significant temporal information. It can extract local features and in 1D convolutions it has some ability to traverse backwards through time but not to any significant degree. As seen in section 3.1.1, the accumulation of drift is due to the integration of the angular rate as we move forward in time.

6.2.3 Temporal Convolutional Networks (TCNs)

TCNs are an extension of the baseline CNN and include two main differences in their implementation as shown by Bai et al.[39]. These differences are the use of dilations that grow the receptive field allowing the network to look back further in time and learn long-term information. Furthermore, causal convolutions meaning, that the convolution operation cannot look forward in time. To explain causality, we can look back at equation 19 and modify this so

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t-k] \tag{21}$$

$$y_f[t] = \phi(z_f[t]) \tag{22}$$

The difference is that the k is subtracted from $x[t]$. Causality is enforced here as at time t, we cannot access future information i.e. $x[t+1], x[t+2]$... etc.

Dilations allow the model to look further back in time without the need of increasing parameters for the same K and C_{in} used in convolutions. This is best represented in figure 6.

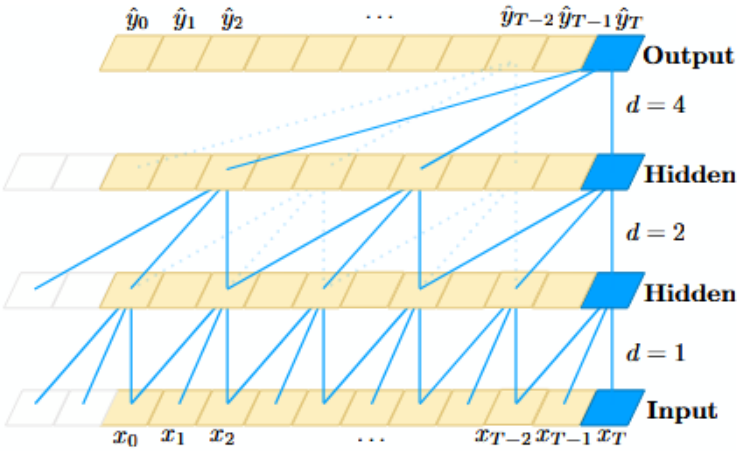


Figure 6: CNN structure with dilations [39]

As we can see in the diagram, as the dilations increase layer by layer, the output to the next layer looks further back into the history of $x[t]$. This can be modelled by the equation

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t-dk] \tag{23}$$

where d is the dilation factor of the layer.

6.2.4 Summary and Choice

In summary, the choice has been to select the TCN. This is because it can look back in history and model long-term dependencies between data, something baseline CNNs cannot do. Additionally, like a CNN it is quicker and more stable in training unlike an RNN. However, the TCN does have its problems, these being extra data storage during its operation and domain transfer. Unlike RNNs which only have store $h(t)$, TCNs need to take in a raw sequence up to its effective history length [39]. Domain transfer refers to transferring a model that has been trained in one domain (i.e. small receptive field) and applied to another domain (i.e. large receptive field), where the TCN may perform poorly due to this change in domain.

6.3 TCN Design

The main design of the TCN comes from Bai et al. [39]. Figure 7 demonstrates the architecture of the residual block that has been implemented in the project.

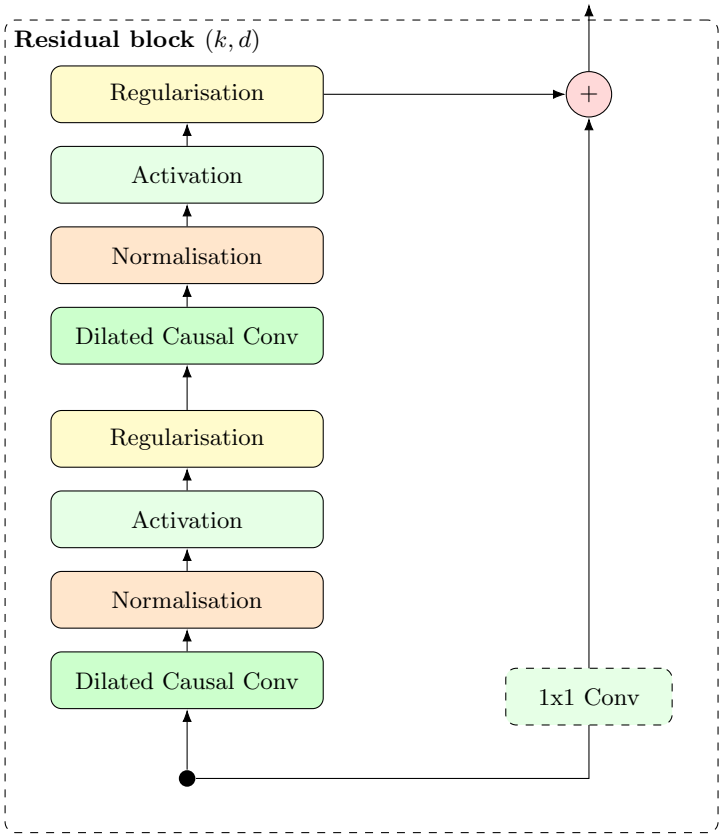


Figure 7: Structure of residual block

The dilated causal convolution is the same as discussed in section 6.2.3.

6.3.1 Normalisation

Normalisation is a technique used in deep learning to ensure training stability, while also reducing training times by allowing the use of higher learning rates. The aim of normalisation is to address internal covariate shift, the change in the distribution of each layer's inputs during training, as parameters of the previous layers change [40]. This can be done by the following techniques.

Method	Normalises	Pros	Cons
BatchNorm	Activations (per channel)	Fast and adds some regularisation	small batch/streaming issues
LayerNorm	Activations (per token/sample)	Batch-size independent	sometimes worse performance in CNNs
GroupNorm	Activations (channel groups)	Works well with small batches	Number of groups need to be selected
WeightNorm	Weights (reparam)	Simple no batches	less regularisation than BatchNorm

Table 6: Common normalisation methods.

From the table above, in training BatchNorm, GroupNorm, and WeightNorm will be tested to see which technique yields the best model accuracy to model training stability. The reason why these have been selected is that BatchNorm has seen wide use in CNN models for drift reduction [34][35]. WeightNorm as the original TCN implementation uses it to avoid errors with small batch sizes [39]. GroupNorm as it is a middle ground where it works well with small batch sizes compared to BatchNorm but it may introduce more complexity with groups needing to be selected and as such will be tested last to see if the other techniques give an acceptable trade-off.

6.3.2 Activation Functions

Activation functions are functions that are used in deep learning to add non-linearity to our output. This is because seen in equations 11,12,13,15,17 the relationship between weights and input data is all linear. The table below demonstrates common activation functions used.

Activation	Definition	Output range	When used	Pros	Cons
ReLU	$\text{ReLU}(x) = \max(0, x)$	$[0, \infty)$	CNN default baseline	Fast activations	ReLU can result in 0 output (dying ReLU)
Leaky ReLU	$\max(\alpha x, x)$, $\alpha \in (0, 1)$	$(-\infty, \infty)$	When ReLU units die	Reduces dying ReLU	extra hyperparameter α
GELU	$x\,\Phi(x)$ (Gaussian CDF)	$(-\infty, \infty)$	Transformers, deep networks	Smooth and good performance	slower than ReLU
Swish	$x\,\sigma(\beta x)$	$(-\infty, \infty)$	Transformers /CNNs	Smooth, often improves accuracy	slower than ReLU

Table 7: Common activation functions.

From the table above, the motivation for ReLU is that it is commonly used as the baseline for CNNs and is also used by Bai et al. in their TCN implementation. However, they can be prone to overfitting [34] or dying ReLU can occur. If ReLU is prone to dying ReLU, then leaky ReLU will be used and tested against the other functions. GELU and Swish are known to be smooth implementations of ReLU, where it is able to take values where $x < 0$ and can have slight performance improvements [41] but are computationally more expensive and performance is model dependent. For implementation, all activation functions will be tested (see section 6.4.1).

6.3.3 Regularisation

Regularisation is a technique used in deep learning models to help models avoid overfitting to training data and allows the model to be more generalisable to unseen data. Overfitting is when the model learns its training data too precisely, so it performs well to training data but less than adequate to unseen data.

The technique that will be implemented first is dropout, where dropout randomly zeroes out all activations within the model for an input in training data. If the model is still prone to overfitting, more advanced regularisation techniques may be employed like weight decay which penalises large weights within kernels to allow for smoother functions.

6.3.4 Residual Connections/1x1 Convolution

The right branch (figure 7) is essential in training very deep neural networks where networks may not be able to converge due to exploding/vanishing gradients. This branch is called a residual connection, where the equation below shows its operation

$$y = \mathcal{F}(x, W_i) + Wx \tag{24}$$

where y is the output, $\mathcal{F}(x, W_i)$ is the residual mapping to be learned, W, is a projection applied to x (1x1 convolution), and x is our input[42].

Residual connections add inputs of a block to the output of the block, so when learning the model learns the relationship of $\mathcal{F}(x, W_i) = y - Wx$, which is easier to learn than the full mapping directly. The 1x1 convolution is included as the model goes deeper, the number of channels increases to capture more complex features as each channel has their own set of filters/kernels. The addition between $\mathcal{F}(x, W_i)$ and x must be of two tensors of the same size so W allows the addition to be possible.

6.4 Hyperparameter Optimisations

The motivation for hyperparameter optimisation is that with the right hyperparameters the model performs significantly better than a model with different hyperparameters. Due to the number of different hyperparameters, being architecture parameters and training parameters, the search space would be too large for methods like grid or random search. Grid search, searches the entire space which would be very inefficient and waste compute, whilst random search with a search space this large would also be inefficient. Moreover, these hyperparameters also interact with each other in non-trivial ways, meaning a certain hyperparameter cannot be fixed as for that current setup it may be the best performer but for another set of hyperparameters it may be worse.

Thus, it was decided to employ use Optuna [43], hyperparameter optimisation framework that can use Bayesian optimisation to efficiently search for the best hyperparameters.

6.4.1 Search Space

Hyperparameter	Type	Values / Range	Notes
`n_layers`	Integer	[2, 7]	Number of TCN layers
`base_channels`	Categorical	{8, 16, 24, 32}	Base channel width
`channel_growth`	Categorical	{`double`, `constant`, `gradual`}	`double`: exponential; `constant`: 4× base; `gradual`: linear to 8× base
`dilation`	Integer	[2, 4]	Dilation base factor
`kernel_size`	Integer	$[k_{\min}, k_{\max}]$	Constrained per trial; receptive field $\in [285, 1023]$; clamped to [3, 9]
`norm_type`	Categorical	{`batchnorm_only`, `weight_norm_only`, `batchnorm_weight_norm`}	—
`activation`	Categorical	{`relu`, `gelu`, `leaky_relu`, `swish`, `mish`}	—
`dropout`	Float	[0.0, 0.4]	Step size 0.05
`lr`	Float (log)	$[10^{-5}, 2 \times 10^{-3}]$	Log-uniform sampling
`weight_decay`	Float (log)	$[10^{-4}, 10^{-1}]$	Log-uniform sampling
`scheduler_type`	Categorical	{`CosineAnnealingLR`, `CosineAnnealingWarmRestarts`, `OneCycleLR`}	—

Table 8: Optuna search space for the TCN residual architecture and training hyperparameters.

The total discrete search space is $136,080$, whilst the true search space is near infinite due to learning rate and weight decay being unbounded. This is the reason as to why grid or random search would be inefficient. The search space is also conditional to not include receptive fields larger than 1024 (3.58s).

6.4.2 Tree Structured Parzen (TPE) Sampler

TPE models the distribution of hyperparameter configurations based on whether the loss falls below or above a quantile threshold (y^*) which is set to the top 25% of trials in this project. The main equation that governs TPE is.

$$p(x \mid y) = \begin{cases} \ell(x) & \text{if } y < y^* \\ g(x) & \text{if } y \geq y^* \end{cases} \tag{25}$$

where $\ell(x)$ and $g(x)$ are kernel density estimates of the 'good' and 'bad' trials respectively [44].

The piecewise function above states how the model changes the probability of a hyperparameter configuration based on whether a trial lands below or above the quantile threshold. On the next iteration of

the proposed configuration, TPE aims to maximise the ratio of $\ell(x)/g(x)$ and on each iteration returns the candidate with the greatest Expected Improvement (EI) [44].

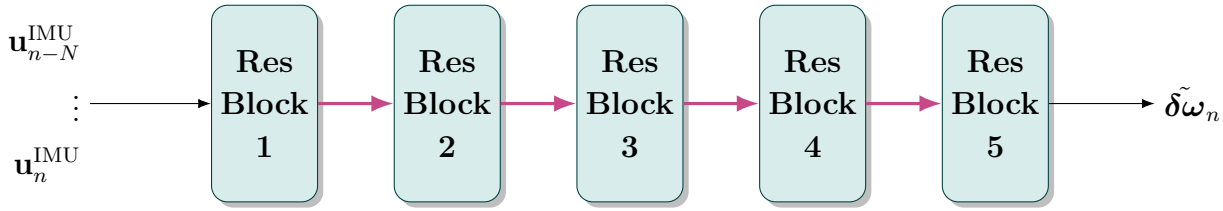
TPE was chosen over other Bayesian optimisation processes like Gaussian Process (GP) was for a few reasons. First, GP scales cubically with observation history, this means as more trials are completed the complexity of GP scales by $O(n^3)$ [44], which can lead to needing more compute compared to TPE. Second, due to the nature of the search space which has different types (see table 8), TPE is able to model these, whilst GP is not able to model categorical types. Third, TPE allows for an option 'multivariate=True' [45], which allows TPE to model the dependencies between the hyperparameters. This is especially relevant for this project as the search space is conditional, this option allows TPE to explore and find the boundaries for acceptable hyperparameters.

6.4.3 HyperBand (HB) Pruner

HB is an early-stopping pruner designed to limit the compute budget wasted on poorly performing trials [46]. HB is based on the successive halving alogrithm, which takes a set of hyperparameter configurations, and trains them on a small budget. It then discards a half of the trials that were the worst performers. Successive halving then doubles the budget of the remaining trials and repeats this for till a winning configuration is found. The problem with this approach is, if the set is small, less configurations are explired. If the set is large each configuration get less training time which may not be enough epochs to determine a clear winner [46].

HB overcomes this limitation by using multiple successive halving 'brackets' that explore a different set of configurations, each bracket have different budgets and set of configurations. One bracket may have a larger set and smaller budget and vice versa. The best result across all brackets is returned. Li et al. mentioned that a 50 times maximum speedup compared to random search is achievable [46]. This is due to the savings that HB offers by pruning bad trials earlier than allowing bad configurations train to the maximum budget. Across a large search space, like this project, these savings compound exponentially as worse performers are pruned early.

6.5 Final Model Configuration



Res. block #	1	2	3	4	5
Kernel size k	7				
Dilation 2^i	1	2	4	8	16
Channels	96				
Dropout	0.15				
Activation	GELU				
Normalisation	Batch Normalisation				

Figure 8: Final Structure of the TCN

The results of the optuna trial returned with the model strucuture laid out above. The search also found the optimal training parameters as being, learning rate = 0.00198, weight decay = 0.000174, and the scheduler of OneCycleLR.

6.6 Proposed Solution

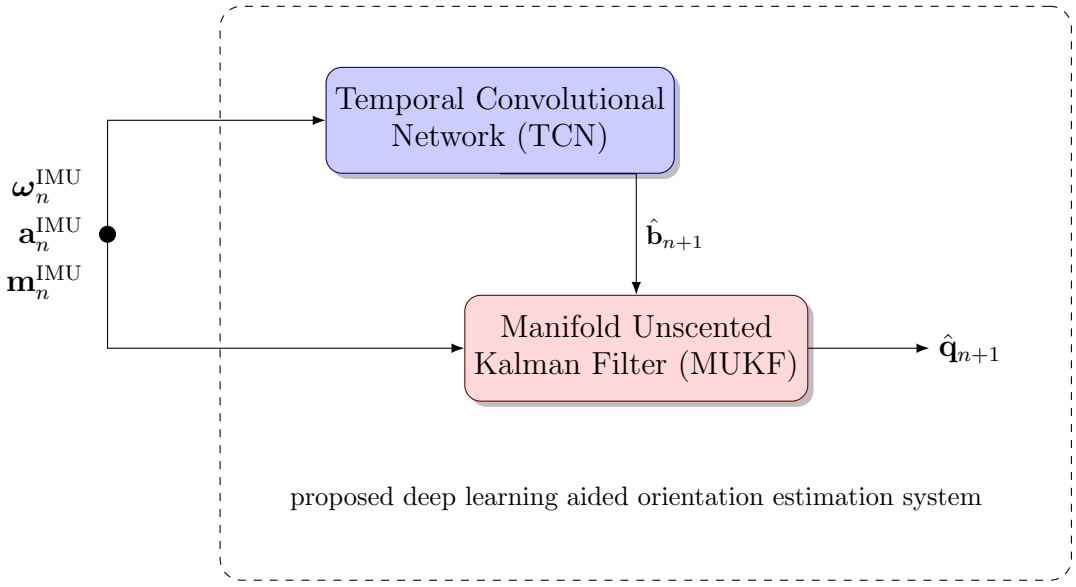


Figure 9: Final Structure of the Proposed Solution

7 Loss Function Design

A loss function in deep-learning is a function used to calculate an error metric, that determines the error between a model's current prediction compared to labelled data. This is supervised training, where the model can use this labelled ground-truth data to determine how incorrect its prediction is and through backpropagation can update its weights to try and minimise this loss. The loss function for this project works by supervising the output of the model, which is a gyroscopic correction applied to the raw gyroscope measurement, then by comparing this output to the ground truth to calculate a loss between the two. Due to the nature of manifolds (see section 5.3), a simple comparison between the output and the ground truth is invalid as the output is a correction and the ground truth is quaternions, meaning a specific loss function has to be designed to overcome these limitations.

7.1 Pairwise Reduction

The first step in the loss function is to take the prediction made from the model and integrate it using the equation 3, this results in a window of relative quaternions from the output of the model.

The next step is pairwise reduction of the relative quaternions. The idea behind the pairwise reduction is that we need to compose long windows of rotations together to be used at different decimation points (see section 7.2). First, we compute the number of reductions that need to happen for a window size of T. This is calculated by

$$steps = \lceil \log_2(T) \rceil \tag{26}$$

From steps, we loop from 0 to steps, where i is the index. We can then calculate our stride with

$$stride = 2^i \tag{27}$$

Next, each element (t) in tensor is multiplied by the element at $t - stride$ using the Hamiliton product. This means that in each iteration there is one Hamiliton product per element per iteration. This algorithm repeats until the loop reaches *steps* and returns a tensor of the same size T, where each element in the tensor now represents a cumulative orientation from the start of our prediction. The tensor is also normalised after every step due to repeated Hamiliton products can accumulate floating-point drift. The figure below shows a graphical view on how this algorithm works.

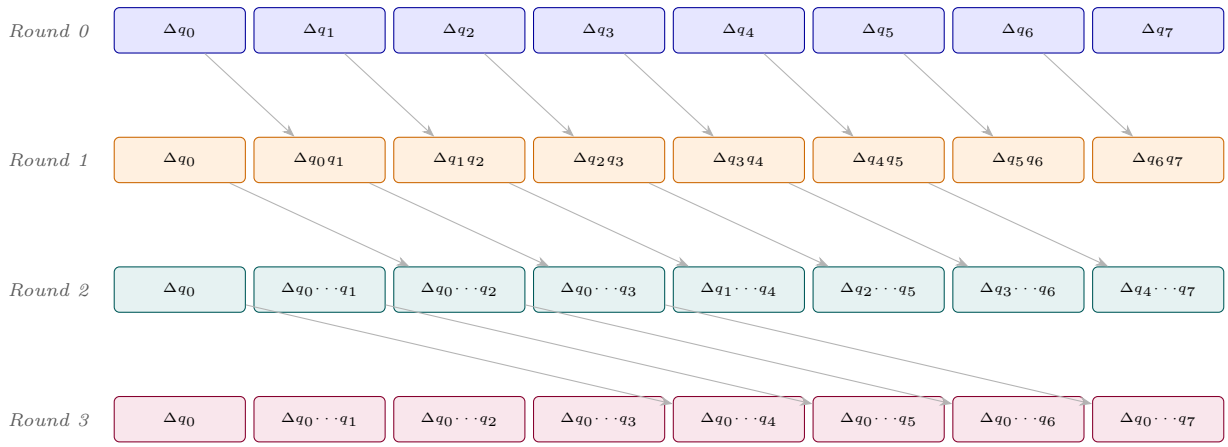


Figure 10: Example of Pairwise Reduction with $T = 8$

The reason as to why pairwise reduction was implemented was that it is a computationally efficient way of producing the cumulative orientations. The same result could have been achieved by composing each element in the tensor sequentially. This works for small windows and batch sizes, $T = 2018$, so the depth in the pairwise approach is only $\lceil \log_2(2018) \rceil = 11$, compared to sequential which would be $T - 1 = 2018 - 1 = 2017$. This approach alone vastly reduced the training time for the model. The reason

as to why this is more efficient, is that all Hamiliton products are calculated in parallel due to Graphical Processing Units (GPUs) ability to parallelise calculations. This means that depth is what represents the limiting factor is calculation time.

7.2 Multi-Scale Decimated Loss

Decimation is the process of selecting a subset of points from a window at a given interval. For the loss function this is calculating the loss at different timescales, such that we can capture small and long range drift effects across the window. This is vital as if the model were to only see one timescale, the model would only be able to capture the effect of drift at that timescale. The consequence of not supervising at different timescales, is that these unsupervised scales are not penalised during training, thus the model has no training signal to be able to correct it. The number of scales is derived from

$$scales = \lceil \log_2(T/baseInterval) \rceil \tag{28}$$

where $baseInterval$ is selected to be 16 empirically. Looping over the number of scales we calculate the interval for the subsample to be

$$interval = baseInterval * 2^i \tag{29}$$

where i is the index of the loop.

However, the model should not be penalised on predictions on where it does not have full context, as this would unfairly scale losses to be dominated by the first few samples. Thus, we employ as method of only counting decimation intervals once the model has filled its receptive field.

The decimation also pairs well with section 7.1, this is because as the relative quaternions have already been computed at each timescale, we can extract the points the loss function wants with one tensor operation. We similarly do the same process of reduction on the ground-truth quaternions to find the quaternion that maps one ground-truth quaternion to another.

Before computing the error, the function first ensures that the decimated prediction quaternions and ground-truth quaternions are both sign-aligned. This is due to $\mathbf{q}$ and $-\mathbf{q}$ representing the same rotation on SO(3). If this was not implemented, the loss function would incorrectly penalise predicitons that represent the same rotation.

After this we compute the quaternion error between the ground-truth and prediction by performing the Hamiliton product. The function then decomposes this quaternion error into a rotational vector in local tangent space using equation 14, where the loss function minimises the error using Huber loss. The reason as to why the loss function uses Huber loss compared to mean squared error (MSE) or mean absolute error (MAE), is that it makes the loss function more robust to larger orientation errors the model predicts in earlier epochs.

Finally, we scale the loss by a factor of $1/2^k$, this means that finer scales are weighted more compared to coarser scales. The reason behind this is at coarser scales the loss function supervises over longer timescales where drift compounds and errors are larger. Thus, without the weighting coarser scales would dominate the loss signal. The equation below summarises the entire loss function.

$$\mathcal{L} = \sum_{k=0}^{K} \frac{1}{2^k} \mathcal{L}_{\text{Huber}}\Big(\mathbf{0},\, \log_{\text{SO(3)}}\Big(\mathbf{q}_{gt,k}^{-1}\, \hat{\mathbf{q}}_k\Big)\Big) \tag{30}$$

8 Results

8.1 Evaluation Metrics

Multiple metrics are used to evaluate the model performance, here in this section we outline each metric and give a motivation as to why. Throughout the results and evaluation both metrics are reported together. The reason as to why both metrics are reported is that even though the geodesic angle error might decrease the yaw error may increase.

8.1.1 Geodesic Angle Error

To determine the error between the two quaternions that exist as rotations on SO(3), we use the geodesic rotation angle between the two quaternions. This metric measures the minimal rotation needed to align the two quaternions.

$$\epsilon_{\text{geo}} = 2\arccos\big(|\hat{q} \cdot q_{\text{gt}}|\big) \tag{31}$$

The reason as to why we use the geodesic angle error is that it is the correct distance between two quaternions on SO(3). A Euclidean difference is not rotation invariant meaning q and $-q$ would represent different rotations, where they are the same rotation on SO(3). It also captures the total orientation error (roll, pitch, and yaw) between the predicted orientation and the ground truth.

8.1.2 Absolute Yaw Error

The equations below show how to calculate the absolute yaw error (AYE).

$$\psi = \arctan 2\big(2(wz + xy),\ 1 - 2(y^2 + z^2)\big) \tag{32}$$

$$\epsilon_{\text{yaw}} = \begin{cases} |\psi_{\text{pred}} - \psi_{\text{gt}}| & \text{if } |\psi_{\text{pred}} - \psi_{\text{gt}}| \leq 180^\circ \\ 360^\circ - |\psi_{\text{pred}} - \psi_{\text{gt}}| & \text{otherwise} \end{cases} \tag{33}$$

The yaw is extracted from the quaternion rotation before the use of the equation above. Further, the AYE is wrapped over [0, 180], such that errors near $\pm 180^\circ$ are not artificially inflated.

The reason as to why the AYE is an interesting metric to track is because of section 3.3. In this section it is mentioned that only the magnetometer can provide a heading reference and thus with corrupted magnetometer measurements this drift in yaw becomes unbounded. By specifically looking at AYE, we can see how much the proposed solution is able to correct yaw especially due to the test set conditions in section 4.1.3.

8.2 Test Pipeline

There are two different pipelines tested within this project. First, is dead reckoning (gyroscope integration only), this pipeline tests how the TCN performs by only providing ω corrections which is applied additively to the raw gyroscope measurement. This test shows how the TCN performs solely as a gyroscope corrector. Second, is the full proposed solution with the integration of the MUKF. This test shows the TCN performance of the corrected gyroscope measurement in the predict step of the MUKF, whilst corrections applied from the accelerometer and magnetometer in the update step in the MUKF. Both of these pipelines are compared to their raw gyroscope counterparts.

The rest of the results section will cover these two pipelines and report the per file statistics and the total aggregated statistics over the entire test set.

8.3 Aggregated Results

Tier	Pipeline	Geodesic Error (°)				Absolute Yaw Error (°)			
		Mean	Std	Median	P90	Mean	Std	Median	P90
Dead Reckoning	Raw Gyro	71.389	43.427	68.509	131.843	50.813	43.401	40.696	117.861
	TCN Gyro	44.291	31.579	38.393	89.377	30.030	30.796	19.115	79.844
	Improvement	***+38.0%***	***+27.3%***	***+44.0%***	***+32.2%***	***+40.9%***	***+29.0%***	***+53.0%***	***+32.3%***
Sensor Fusion	Raw Gyro + MUKF	51.851	41.238	40.995	116.735	43.897	41.808	29.831	110.757
	TCN + MUKF	31.749	25.319	24.823	68.692	26.593	27.742	17.327	61.966
	Improvement	***+38.8%***	***+38.6%***	***+39.4%***	***+41.2%***	***+39.4%***	***+33.6%***	***+41.9%***	***+44.1%***

Table 9: Aggregated orientation error across all five test trials (mean of per-trial means). Geodesic angle error (ϵ_{geo}) and absolute yaw error (ϵ_{yaw}) in degrees. Improvement rows show percentage reduction in error relative to the raw gyroscope baseline within each tier.

8.4 Trial 9: Undisturbed, Fast Rotation

Tier	Pipeline	Geodesic Error (°)				Absolute Yaw Error (°)			
		Mean	Std	Median	P90	Mean	Std	Median	P90
Dead Reckoning	Raw Gyro	75.371	58.073	67.612	162.423	37.430	51.008	14.359	131.874
	TCN Gyro	56.805	41.942	49.745	112.683	44.059	44.654	29.110	123.329
	Improvement	***+24.6%***	***+27.8%***	***+26.4%***	***+30.6%***	***−17.7%***	***+12.5%***	***−102.7%***	***+6.5%***
Sensor Fusion	Raw Gyro + MUKF	46.454	52.109	17.393	138.242	43.220	54.157	10.494	137.716
	TCN + MUKF	29.159	23.530	23.388	62.434	28.171	35.502	16.241	60.062
	Improvement	***+37.2%***	***+54.8%***	***−34.5%***	***+54.8%***	***+34.8%***	***+34.4%***	***−54.8%***	***+56.4%***

Table 10: Orientation error for trial 9 (undisturbed, fast rotation with breaks).

8.5 Trial 25: Disturbed Tapping

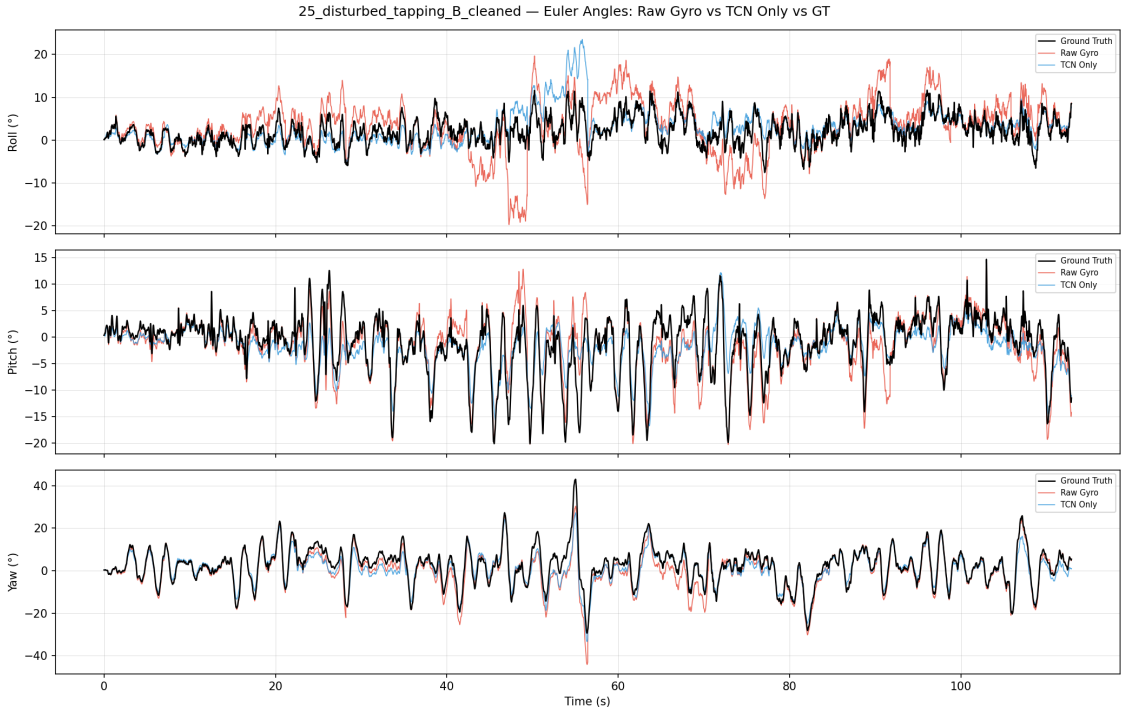


Figure 11: Euler Angle Plots (dead reckoning) for Trial 25: Disturbed Tapping

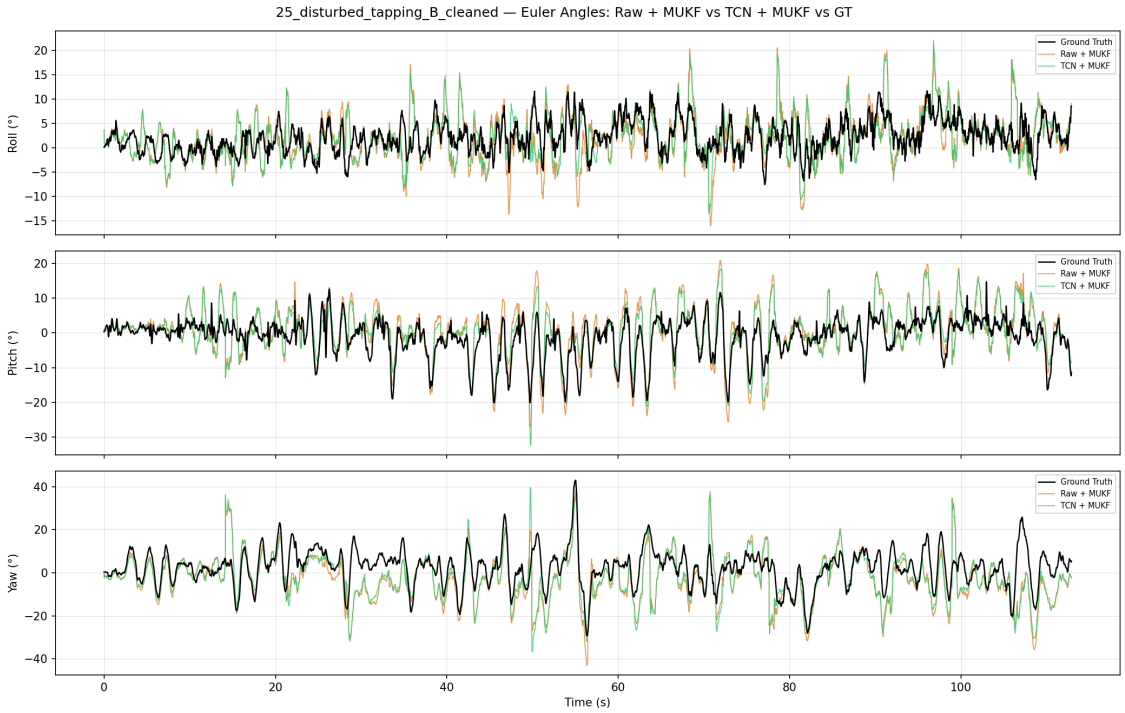


Figure 12: Euler Angle Plots (MUKF pipeline) for Trial 25: Disturbed Tapping

		Geodesic Error (°)				Absolute Yaw Error (°)			
Tier	**Pipeline**	Mean	Std	Median	P90	Mean	Std	Median	P90
Dead Reckoning	Raw Gyro	5.381	4.027	4.441	9.767	2.400	2.680	1.471	5.835
	TCN Gyro	4.657	3.297	3.960	8.526	2.380	2.308	1.696	5.581
	Improvement	***+13.5%***	***+18.1%***	***+10.8%***	***+12.7%***	***+0.8%***	***+13.9%***	***−15.3%***	***+4.4%***
Sensor Fusion	Raw Gyro + MUKF	9.892	7.186	8.316	18.698	7.544	7.044	5.156	16.540
	TCN + MUKF	9.545	7.292	7.721	18.380	7.220	7.298	4.981	16.134
	Improvement	***+3.5%***	***−1.5%***	***+7.2%***	***+1.7%***	***+4.3%***	***−3.6%***	***+3.4%***	***+2.5%***

Table 11: Orientation error for trial 25 (disturbed, tapping).

8.6 Trial 31: Disturbed Stationary Magnet

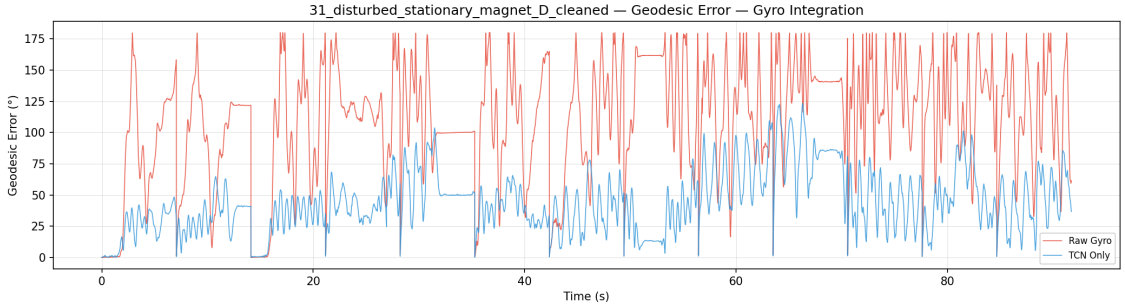


Figure 13: Geodesic Angle Error (dead reckoning) for Trial 31: Disturbed Stationary Magnet

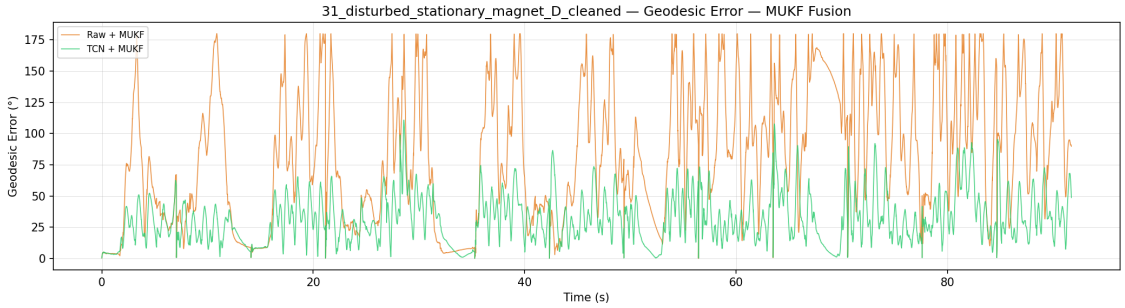


Figure 14: Geodesic Angle Error (MUKF pipeline) for Trial 31: Disturbed Stationary Magnet

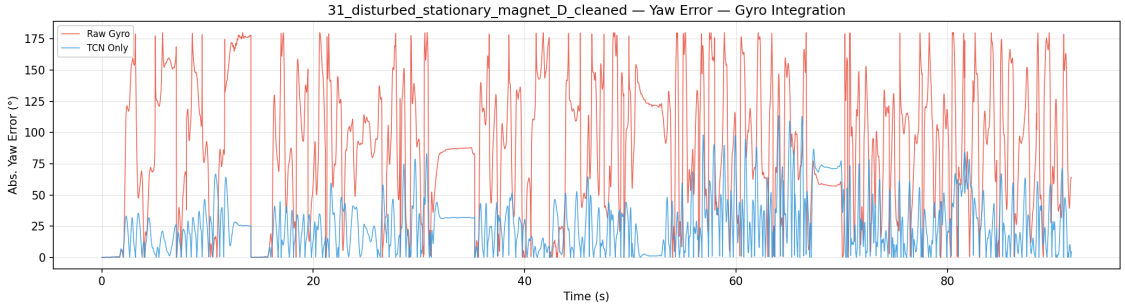


Figure 15: Absolute Yaw Error (dead reckoning) for Trial 31: Disturbed Stationary Magnet

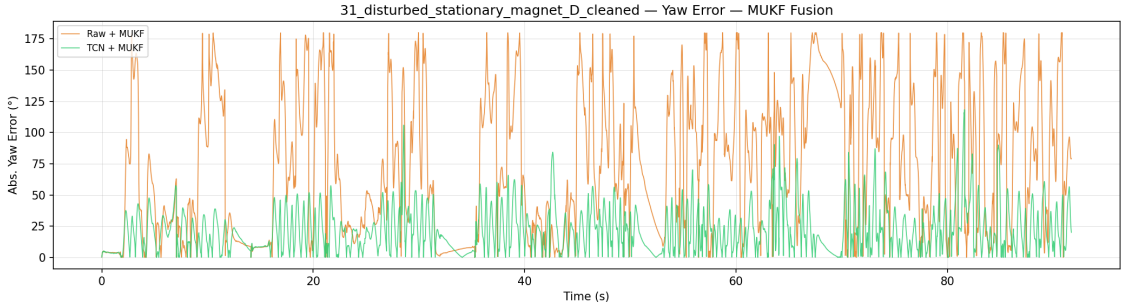


Figure 16: Absolute Yaw Error (MUKF pipeline) for Trial 31: Disturbed Stationary Magnet

		Geodesic Error (°)				**Absolute Yaw Error (°)**			
Tier	**Pipeline**	Mean	Std	Median	P90	Mean	Std	Median	P90
Dead Reckoning	Raw Gyro	111.134	44.795	118.547	163.132	88.799	51.572	88.343	159.099
	TCN Gyro	44.715	24.392	41.784	81.876	25.828	21.212	22.311	56.746
	Improvement	***+59.8%***	***+45.5%***	***+64.8%***	***+49.8%***	***+70.9%***	***+58.9%***	***+74.7%***	***+64.3%***
Sensor Fusion	Raw Gyro + MUKF	85.194	52.420	83.029	161.128	75.227	55.425	66.535	156.699
	TCN + MUKF	31.443	19.537	28.882	58.609	21.563	17.675	17.551	46.356
	Improvement	***+63.1%***	***+62.7%***	***+65.2%***	***+63.6%***	***+71.3%***	***+68.1%***	***+73.6%***	***+70.4%***

Table 12: Orientation error for trial 31 (disturbed, stationary magnet).

8.7 Trial 34: Disturbed Magnet attached 3cm

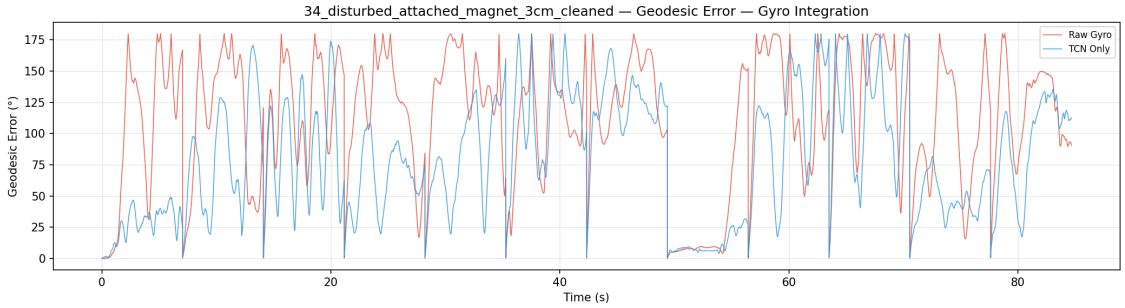


Figure 17: Geodesic Angle Error (dead reckoning) for Trial 34: Disturbed Magnet attached 3cm

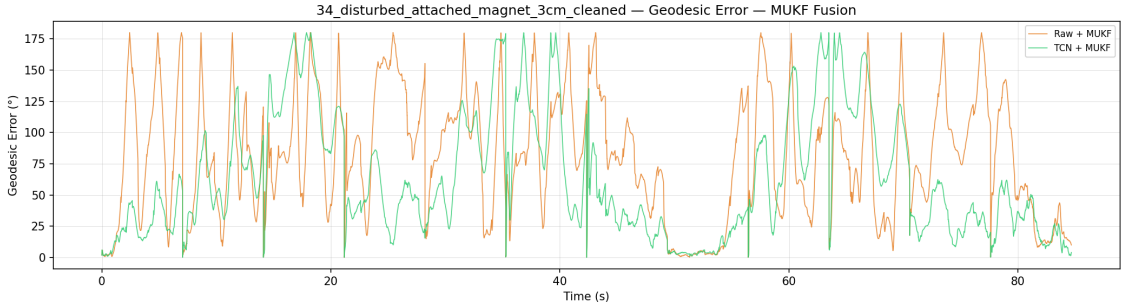


Figure 18: Geodesic Angle Error (MUKF pipeline) for Trial 34: Disturbed Magnet attached 3cm

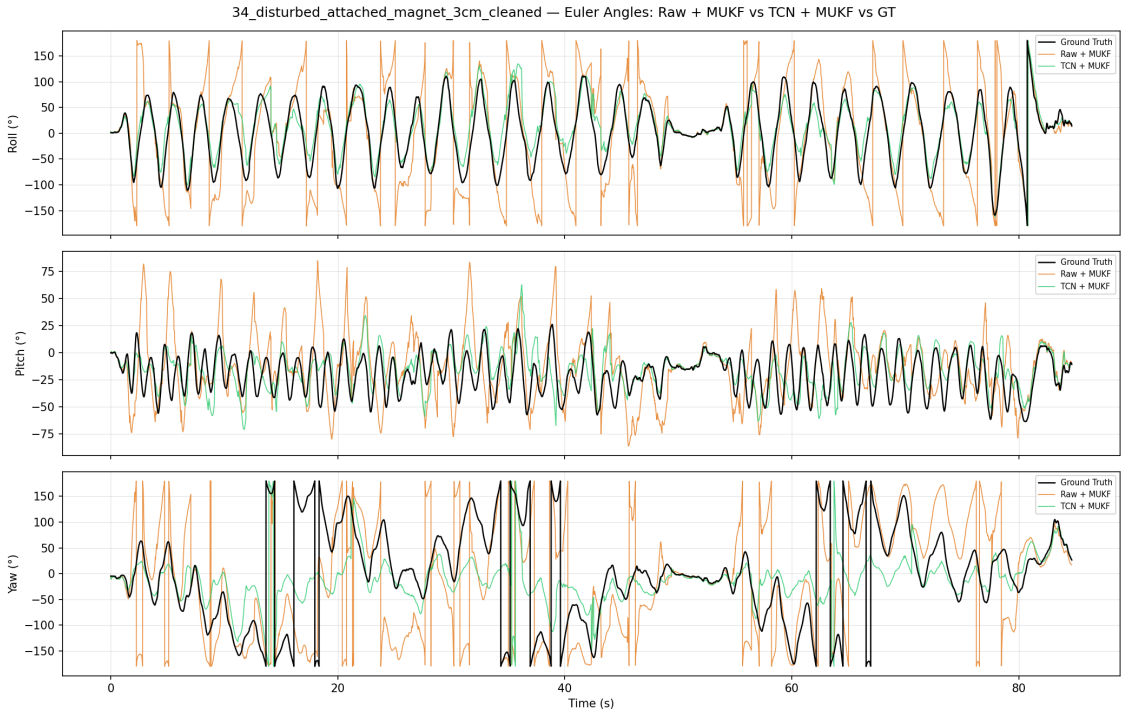


Figure 19: Euler Angle Plots (MUKF pipeline) for Trial 34: Disturbed Magnet attached 3cm

		Geodesic Error (°)				Absolute Yaw Error (°)			
Tier	**Pipeline**	Mean	Std	Median	P90	Mean	Std	Median	P90
Dead Reckoning	Raw Gyro	111.878	51.871	124.173	170.923	84.637	59.990	88.132	164.224
	TCN Gyro	80.055	48.754	77.076	145.770	52.168	45.448	35.869	125.067
	Improvement	***+28.4%***	***+6.0%***	***+37.9%***	***+14.7%***	***+38.4%***	***+24.2%***	***+59.3%***	***+23.8%***
Sensor Fusion	Raw Gyro + MUKF	81.234	48.729	80.444	150.411	62.208	49.122	54.595	135.670
	TCN + MUKF	63.476	48.925	47.692	144.704	55.744	51.321	36.852	138.004
	Improvement	***+21.9%***	***−0.4%***	***+40.7%***	***+3.8%***	***+10.4%***	***−4.5%***	***+32.5%***	***−1.7%***

Table 13: Orientation error for trial 34 (disturbed, magnet attached 3 cm from IMU).

8.8 Trial 39: Disturbed, Mixed Conditions

		Geodesic Error (°)				Absolute Yaw Error (°)			
Tier	**Pipeline**	Mean	Std	Median	P90	Mean	Std	Median	P90
Dead Reckoning	Raw Gyro	53.181	58.371	27.774	152.970	40.797	51.757	11.174	128.271
	TCN Gyro	35.223	39.512	19.399	98.031	25.713	40.358	6.588	88.499
	Improvement	***+33.8%***	***+32.3%***	***+30.2%***	***+35.9%***	***+37.0%***	***+22.0%***	***+41.0%***	***+31.0%***
Sensor Fusion	Raw Gyro + MUKF	36.483	45.747	15.792	115.195	31.284	43.291	12.376	107.162
	TCN + MUKF	25.122	27.312	16.433	59.333	20.265	26.913	11.009	49.274
	Improvement	***+31.1%***	***+40.3%***	***−4.1%***	***+48.5%***	***+35.2%***	***+37.8%***	***+11.0%***	***+54.0%***

Table 14: Orientation error for trial 39 (disturbed, mixed conditions).

The rest of the results graphs can be found in appendix A.

9 Evaluation

Looking at table 9, we can see that the deep-learning model produced significant improvements across all metrics in the entire test set. Both TCN Gyro and TCN + MUKF saw around a 38% improvement in mean geodesic error and 40% mean improvement in AYE, when compared to their raw gyroscope measurement counterparts. These metrics directly support the fact that the TCN model is able to address drift in tilt and heading whilst both accelerometer and especially magnetometer are disturbed, which are the typical failure modes seen in orientation estimation with IMUs.

Furthermore, comparing TCN dead reckoning to raw gyroscope + MUKF, we can see that the TCN performs better, which highlights that the TCN alone does beat uncorrected raw gyroscope measurments even with corrections from an accelerometer and magnetometer. The comparison between the two produces significant results with improvements in mean geodesic error 14.6%, 31.6% in mean AYE, and 35.9% in median AYE. This result is expected as the test set does contain disturbed accelerometer and magnetometer conditions.

Moreover, median improvements in TCN only of 44% in geodesic error and 53% in AYE, and 39.4%, and 41.9% in TCN + MUKF respectively, show a promising result that not only does the TCN produce fewer errors (mean) but also a more consistent output as well. Coupled with the P90 improvements of 41.2% in geodesic error, and 44.1% in AYE, for the TCN + MUKF, these results show that the entire distrubution landscape of errors is tighter. It shows that not only do we have overall better orientation estimates, but these improvements are more consistent and tail errors are supressed.

9.1 Trial 9: Undisturbed, Fast Rotation

Looking at table 10, we can see that overall there have been significant improvements in all geodesic error metrics across the two pipelines, with the TCN + MUKF pipeline performing the best with an improvement of 37.2% in mean geodesic error and the largest improvements in P90 and standard deviation of 54.8%. This means that the model is able to provide meaningful corrections and that it is robust to tail errors. However, in the TCN Gyro pipeline we can see that AYE actually gets worse (-17.7%).

Trial 9 is interesting compared to any other fast rotation in the dataset, this is because this trial exhibits fast rotations predominantly in the z-axis (yaw). This gives the motiviation as to why AYE was chosen as a metric. Specifically 50% of Trial 9 rotations are in the z-axis, whilst fast rotation training trials incorporated 67% of its rotations in the x-axis (roll). This may be the reason to why in dead reckoning the TCN Gyro actually suffers and performs worse for AYE compared to the raw gyro. This also explains why the overall geodesic error for the TCN Gyro is better than raw gyro, the TCN is able to learn and correct from the trials with fast rotations in other axes, however when the trial is now predominantly in the z-axis its corrections that it applies is not enough for this specific trial. Therefore, it can be concluded that the TCN generalises well for fast rotations, however it is sensitive to the axis composition of the rotation in testing. This is a generalisation boundary for the TCN, but it does still apply corrections to the point that there is a general increase in performance.

Trial 9 is also good motivation and proof for the full proposed solution compared to just dead reckoning. As the TCN was limited by its training data, the MUKF was able to guide and correct the AYE by providing magnetometer updates to correct the heading drift. This shows that deep-learning should be used in tandem with Bayesian filtering and not replace it, as the combination of the two is able to compensate for when one of these systems is underperforming.

9.2 Trial 25: Disturbed Tapping

Trial 25 is the most interesting trial as it exhibits the lowest overall geodesic error and AYE of any other trial, whilst subverting expectations for the proposed solution. In this test the largest errors that are seen is a mean geodesic error of 9.892° and a AYE mean of 7.544° (raw + MUKF). This means overall within the trial there was limited drift over its entirety and this can be explained through what the trial entails. The trial is described as medium-speed translational movements without rotation, no breaks, tapping with finger [15]. The trial explains the small error as the absence of rotation limits drift. Additionally, it can be seen that in both pipelines that the TCN provides an improvement be it a very small improvement.

First, both MUKF pipelines performed worse than the dead-reckoning counterparts. This can be explained through the update step of the MUKF, the high-frequency tapping motions experienced caused

corrupted accelerometer measurements to propagate through the filter and provide incorrect updates to the filter. It also explains the increase in AYE error as well, as the yaw correction applied by the magnetometer assumes that the accelerometer update has corrected any tilt drift, as the magnetometer correction is rotated into the body frame using the tilt. In this exact trial, it can be seen that the MUKF is the main source of error compared to the TCN, this can be seen in the figures 11 and 9 . Specifically, figure 12 at $t \approx 45s - 55s$, shows that yaw is being somewhat accurately tracked, but roll/pitch estimation is degraded and this was under a burst of tapping.

This trial draws an important conclusion for the proposed solution, that this approach whilst better and robust to most conditions, specific high-frequency vibrations are a failure point. This is because any accelerometer and magnetometer dynamics are tied to the filter to correct whilst the TCN only focuses on gyroscope measurements. With a underperforming gate (section 5.1.2) on the accelerometer, the TCN is not able to address this and ultimately is corrupted by the filter. However, the improvements we saw in each respective pipeline does still support that the deep-learning model aided in the reduction of drift.

9.3 Trial 31: Disturbed Stationary Magnet

Trial 31 showed the strongest results with all metrics improving across both pipelines, a mean geodesic error improvement of 59.8% and 63.1% and a mean AYE improvement of 70.9% and 71.3% for TCN Gyro and TCN + MUKF respectively. Improvements appear not only in mean but across all reported metrics, this shows that the proposed solution estimates were more accurate, robust, and consistent across the entire trial. This is also supported by the figures 13 - 16.

The MUKF alone underperforms here due to the nature of the trial. The trial is described as medium-speed combined rotational/translational motions, four breaks of 5 seconds, small magnet placed in stationary location [15]. A stationary magnet here adds a constant hard iron effect (see section 3.3), to the system and the rotational and translational motions also show that the effect varies spatially. This means that every heading correction the MUKF applies is consistently in the wrong direction. Even with the magnetometer gate rejecting the corrupted measurements, the filter cannot recover heading. The heading should drift as there is no magnetometer to correct it, but the TCN shows its value here by keeping heading drift contained, this is directly the gap that the TCN needs to fulfil and does.

9.4 Trial 34: Disturbed Magnet attached 3cm

Trial 34 differs from 31 in that the magnetic disturbance now varies temporally. This is because the magnet is fixed at 3cm from the IMU but as the IMU undergoes any motion, the incident magnetic disturbance changes over time. This is a more challenging disturbance to try and correct as the TCN cannot learn a stable correction pattern as the disturbance changes over time. This is also supported by the table 13 as the improvement percentage for mean geodesic error decreases to 28.4% and 21.9% and mean AYE improvement decreases to 38.4% and 10.4%. However, it should be noted that overall in most metrics the TCN does provide improvements to the system. The figures 17 and 18 also highlights the difficulty in this trial as errors cycle between 0° to 175°, showing that all pipelines struggled in achieving accurate orientation estimates.

This trial also shows a limitation of the proposed solution. As seen in table 13, the mean AYE in the dead reckoning pipeline for TCN Gyro achieves 52.168°, whilst TCN + MUKF achieves 55.744°. Overall the explaination is the same as section 9.2 but instead of corrupted accelerometer updates, it is corrupted magnetometer updates. A similar process happens that when the MUKF, receives magnetometer measurements, some of these corrupted measurements pass through the magnetometer gate (table 5), which causes an incorrect update, causing yaw to drift. This is supported in the figure 19, which shows TCN + MUKF correctly tracks roll and pitch accurately compared to the ground truth and yaw drifts.

This trial exposes the clearest case where this TCN hits its ceiling. As the TCN only addresses the gyroscope, some corrupted measurements are able to get through the gate and decrease its accuracy. This is due to the nature of the disturbance, as a motion correlated magnetometer disturbance is harder to gate for compared to a static bias in (section 3.3). However, like section 9.2, the solution provides overall improvements when compared to dead reckoning and filters. A solution that extends to addressing magnetometers would likely be more effective and provide better improvements.

9.5 Trial 39: Disturbed, Mixed Conditions

Trial 39 is the closest trial to a real deployment scenario that the IMU may experience. It is a combination of slow/fast isolated and combined rotations/translations, with breaks, includes various magnetic disturbances [15]. This trial combines all types of failure modes which is fast rotations and movements, which allows for gyroscope drift to accumulate, accelerometer measurements to no longer give a reference to gravity, and magnetic disturbances which corrupts the heading reference. With this breadth of disturbance types combined, this trial is the most demanding test of generalisation since it combines all disturbance modes simultaneously rather than in isolation.

All metrics significantly improve within this trial for both TCN and TCN + MUKF except the median geodesic error for the TCN + MUKF (-4.1%). This suggests that both pipelines overall provide more accurate, robust and the TCN gyro is more consistent, whereas TCN + MUKF is marginally less consistent. However, for AYE both pipelines improve significantly across all metrics. Therefore it can be concluded that in a trial that is representative of real deployment conditions that the proposed solution offers vast improvements over just traditional filtering methods.

9.6 Summary

In summary, to address the research questions (section 1.3), the deep-learning method is able to learn drift patterns in IMU data to produce drift-reduced gyroscope measurments. Trials 39, 31, and table 9 directly support this conclusion as nearly all metrics saw significant improvements. However, the trials 25 and 34, show the ceiling of this approach, very high frequency vibrations and complex magnetic disturbances are still a challenge even though the solution does provide an improvement over the baseline.

The solution is also able to generalise well, the test set was completely unseen by the model until the best model was selected from validation loss. Trial 9 shows the limitations of the generalisation where the model was able to learn fast rotations but was sensitive to which axis exhibited these rotations.

The correction target was scoped to gyroscope residuals, which gave consistent drift reduction but left accelerometer and magnetometer update errors unaddressed. A solution where it is able to combine this gyroscope correction and address the accelerometer and magnetometer measurements may see a larger increase in performance.

10 Conclusion

References

[1] A. Filippeschi, N. Schmitz, M. Miezal, G. Bleser, E. Ruffaldi, and D. Stricker, "Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion," *Sensors*, vol. 17, p. 1257, Jun. 2017. DOI: 10.3390/s17061257.

[2] R. Kianifar, V. Joukov, A. Lee, S. Raina, and D. Kulić, "Inertial measurement unit-based pose estimation: Analyzing and reducing sensitivity to sensor placement and body measures," *Journal of Rehabilitation and Assistive Technologies Engineering*, vol. 6, p. 205 566 831 881 345, Jan. 2019. DOI: 10.1177/2055668318813455. [Online]. Available: https://journals.sagepub.com/doi/full/10.1177/2055668318813455.

[3] L. Franco, R. Sengupta, L. Wade, and D. Cazzola, "A novel imu-based clinical assessment protocol for axial spondyloarthritis: A protocol validation study," *PeerJ*, vol. 9, e10623, Jan. 2021. DOI: 10.7717/peerj.10623. (visited on 03/14/2022).

[4] Z.-Q. Zhang and J.-K. Wu, "A novel hierarchical information fusion method for three-dimensional upper limb motion estimation," *IEEE Transactions on Instrumentation and Measurement*, vol. 60, pp. 3709–3719, Nov. 2011. DOI: 10.1109/tim.2011.2135070. (visited on 11/27/2021).

[5] G.-Z. Yang, *Body Sensor Networks*. London Springer, 2014.

[6] N. Trawny and S. Roumeliotis, *Indirect kalman filter for 3d attitude estimation*, 2005. [Online]. Available: https://mars.cs.umn.edu/tr/reports/Trawny05b.pdf (visited on 01/09/2026).

[7] O. J. Woodman, "Number 696 an introduction to inertial navigation," *An introduction to inertial navigation*, 2007. [Online]. Available: https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf.

[8] "How good is your gyro [ask the experts]," *IEEE Control Systems*, vol. 30, pp. 12–86, Feb. 2010. DOI: 10.1109/mcs.2009.935122. (visited on 08/31/2021).

[9] M. Looney, *Designing for low noise feedback control with mems gyroscopes*, Analog.com, Mar. 2016. [Online]. Available: https://www.analog.com/en/resources/technical-articles/low-noise-feedback-control-mems-gyroscopes.html (visited on 01/09/2026).

[10] A. M. Sabatini, "Estimating three-dimensional orientation of human body parts by inertial/magnetic sensing," *Sensors*, vol. 11, pp. 1489–1525, Jan. 2011. DOI: 10.3390/s110201489.

[11] *Detection of static and dynamic activities using uniaxial accelerometers — ieee journals & magazine — ieee xplore*, ieeexplore.ieee.org. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=547939.

[12] E. Bachmann, X. Yun, and C. W. Peterson, "An investigation of the effects of magnetic variations on inertial/magnetic orientation sensors," Jan. 2004. DOI: 10.1109/robot.2004.1307974. (visited on 08/02/2023).

[13] J. Luiz, G. H. Elkaim, C. Silvestre, P. Oliveira, and B. Cardeira, "Geometric approach to strapdown magnetometer calibration in sensor frame," vol. 47, pp. 1293–1306, Apr. 2011. DOI: 10.1109/taes.2011.5751259.

[14] *Calibration of a magnetometer in combination with inertial sensors*, Ieee.org, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/6289882 (visited on 01/10/2026).

[15] D. Laidig, M. Caruso, A. Cereatti, and T. Seel, "Broad—a benchmark for robust inertial orientation estimation," *Data*, vol. 6, p. 72, Jun. 2021. DOI: 10.3390/data6070072.

[16] MathWorks, *Navigation toolbox*, Accessed: 2026-05-03, 2026. [Online]. Available: https://uk.mathworks.com/products/navigation.html.

[17] J. Hestness, S. Narang, N. Ardalani, *et al.*, *Deep learning scaling is predictable, empirically*. [Online]. Available: https://arxiv.org/pdf/1712.00409.

[18] Y. LeCun, L. Bottou, G. B. Orr, and K. .-. Müller, "Efficient backprop," *Lecture Notes in Computer Science*, pp. 9–50, 1998. DOI: 10.1007/3-540-49430-8_2. [Online]. Available: https://link.springer.com/chapter/10.1007%2F3-540-49430-8_2.

[19] J. Sola and J. Sevilla, "Importance of input data normalization for the application of neural networks to complex industrial problems," *IEEE Transactions on Nuclear Science*, vol. 44, pp. 1464–1468, Jun. 1997. DOI: 10.1109/23.589532.

[20] Google, *Numerical data: Normalization*, Google for Developers, 2024. [Online]. Available: https://developers.google.com/machine-learning/crash-course/numerical-data/normalization.

[21] P. Contributors, *Pytorch documentation*, Pytorch.org, 2023. [Online]. Available: https://docs.pytorch.org/docs/2.11/index.html.

[22] G. Welch and G. Bishop, *An introduction to the kalman filter*, 2006. [Online]. Available: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf.

[23] E. Gallo, *The so(3) and se(3) lie algebras of rigid body rotations and motions and their application to discrete integration, gradient descent optimization, and state estimation*, arXiv.org, 2022. [Online]. Available: https://arxiv.org/abs/2205.12572 (visited on 05/12/2026).

[24] S. Julier and J. Uhlmann, *A new extension of the kalman filter to nonlinear systems*, 1997. [Online]. Available: https://www.cs.unc.edu/~welch/kalman/media/pdf/Julier1997_SPIE_KF.pdf.

[25] E. Kraft, "A quaternion-based unscented kalman filter for orientation tracking," Jan. 2003. DOI: 10.1109/icif.2003.177425.

[26] M. Brossard, A. Barrau, and S. Bonnabel, "A code for unscented kalman filtering on manifolds (ukf-m)," *HAL (Le Centre pour la Communication Scientifique Directe)*, May 2020. DOI: 10.1109/icra40945.2020.9197489. (visited on 04/17/2024).

[27] J. Crassidis and F. Landis Markley, *Unscented filtering for spacecraft attitude estimation*. [Online]. Available: https://www.acsu.buffalo.edu/~johnc/uf_att.pdf (visited on 05/12/2026).

[28] J. Solà, *Quaternion kinematics for the error-state kalman filter*, arXiv.org, Nov. 2017. DOI: 10.48550/arXiv.1711.02508. [Online]. Available: https://arxiv.org/abs/1711.02508 (visited on 02/23/2024).

[29] Z.-Q. Zhang, X.-L. Meng, and J.-K. Wu, "Quaternion-based kalman filter with vector selection for accurate orientation tracking," *IEEE Transactions on Instrumentation and Measurement*, vol. 61, pp. 2817–2824, Oct. 2012. DOI: 10.1109/tim.2012.2196397. (visited on 09/05/2020).

[30] F. Bo, J. Li, W. Wang, and K. Zhou, "Robust attitude and heading estimation under dynamic motion and magnetic disturbance," *Micromachines*, vol. 14, pp. 1070–1070, May 2023. DOI: 10.3390/mi14051070. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10221362/.

[31] A. Soleimany, *Deep sequence modeling mit 6.s191*, 2019. [Online]. Available: https://introtodeeplearning.com/2019/materials/2019_6S191_L2.pdf (visited on 01/15/2026).

[32] S. Kanai, Y. Fujiwara, and S. Iwamura, *Preventing gradient explosions in gated recurrent units*. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/file/f2fc990265c712c49d51a18a32b39f0c-Paper.pdf (visited on 01/15/2026).

[33] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," *ResearchGate*, Nov. 2012. [Online]. Available: https://www.researchgate.net/publication/233730646_On_the_difficulty_of_training_Recurrent_Neural_Networks.

[34] M. Brossard, S. Bonnabel, and A. Barrau, "Denoising imu gyroscopes with deep learning for open-loop attitude estimation," *IEEE Robotics and Automation Letters*, pp. 1–1, 2020. DOI: 10.1109/lra.2020.3003256. (visited on 06/29/2022).

[35] H. Chen, P. Aggarwal, T. M. Taha, and V. P. Chodavarapu, *Improving inertial sensor by reducing errors using deep learning methodology*, IEEE Xplore, Jul. 2018. DOI: 10.1109/NAECON.2018.8556718. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8556718.

[36] M. Zeiler and R. Fergus, *Visualizing and understanding convolutional networks*, Nov. 2013. [Online]. Available: https://arxiv.org/pdf/1311.2901.

[37] P. V. Hoa, N. A. Binh, P. V. Hong, *et al.*, "One-dimensional deep learning driven geospatial analysis for flash flood susceptibility mapping: A case study in north central vietnam," *Earth Science Informatics*, vol. 17, pp. 4419–4440, Jul. 2024. DOI: 10.1007/s12145-024-01285-8. (visited on 06/12/2025).

[38] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, pp. 84–90, May 2012.

[39] S. Bai, J. Zico Kolter, and V. Koltun, *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling*, Apr. 2018. [Online]. Available: https://arxiv.org/pdf/1803.01271.

[40] S. Ioffe and C. Szegedy, *Batch normalization: Accelerating deep network training by reducing internal covariate shift*, 2015. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.pdf.

[41] P. Ramachandran, B. Zoph, and Q. V. Le, *Searching for activation functions*, arXiv.org, 2017. [Online]. Available: https://arxiv.org/abs/1710.05941.

[42] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, Jun. 2016. DOI: 10.1109/cvpr.2016.90. [Online]. Available: https://ieeexplore.ieee.org/document/7780459.

[43] *Optuna: A hyperparameter optimization framework — optuna 2.10.1 documentation*, optuna.readthedocs.io. [Online]. Available: https://optuna.readthedocs.io/en/stable/.

[44] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems*, J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, Eds., vol. 24, Curran Associates, Inc., 2011. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf.

[45] *Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance*, Arxiv.org, 2026. [Online]. Available: https://arxiv.org/pdf/2304.11127 (visited on 05/19/2026).

[46] L. Li, K. Jamieson, and A. Rostamizadeh, "Hyperband: A novel bandit-based approach to hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 18, pp. 1–52, 2018. [Online]. Available: https://www.jmlr.org/papers/volume18/16-558/16-558.pdf.